

Better, Faster: Harnessing Self-Improvement in Large Reasoning Models

Qihuang Zhong¹ Liang Ding² Juhua Liu³ Bo Du³ Leszek Rutkowski⁴ Dacheng Tao¹

Abstract

Self-improvement training enables the large reasoning models (LRMs) to improve themselves by self-generating reasoning trajectories as training data without external supervision. However, we find that this method often falls short in complex reasoning tasks and even leads to model collapse. Through a series of preliminary analyses, we reveal two problems: (1) *data imbalance*, where most training samples are simple, but the challenging yet crucial samples are scarce; (2) *overthinking*, where many undesired samples with redundant reasoning steps are used for self-training. To this end, we propose **HSIR**, which effectively **Harnesses Self-Improvement** in large **R**easoning models via two simple-yet-effective approaches. Specifically, HSIR introduces a *verify-then-exit* sampling strategy to mitigate data imbalance by efficiently collecting more accurate solutions for difficult queries, and designs an *Intrinsic Diversity* score to quantify overthinking and filter out the undesired solutions. We apply HSIR to various post-training paradigms, among which we further propose H-GRPO, an enhanced GRPO algorithm that leverages the intrinsic diversity as an external reward to encourage concise and diverse reasoning via reinforcement learning. Extensive results show that HSIR not only effectively enhances the reasoning performance, *i.e.*, bringing up to **+10.9%** average performance gains, but also significantly improves the reasoning efficiency by reducing up to **42.4%** relative inference overhead.

1. Introduction

Recently, post-training the large language models (LLMs) with explicit long chain-of-thought (CoT) reasoning trajec-

¹Nanyang Technological University, Singapore ²Alibaba Group, China ³School of Computer Science, Wuhan University, China ⁴AGH University of Science and Technology, Poland. Correspondence to: Dacheng Tao <dacheng.tao@ntu.edu.sg>.

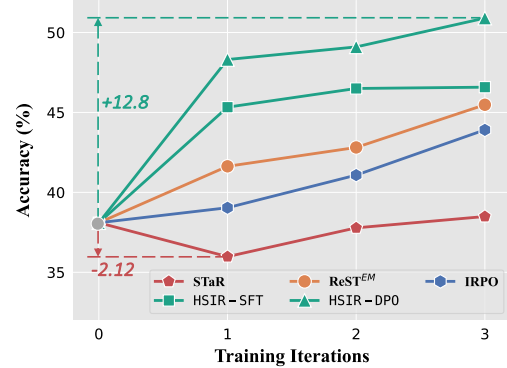


Figure 1. Performance comparison of Qwen2.5-1.5B using different self-improvement training methods on the MedQA task.

tories has garnered significant attention (Li et al., 2025; Plaet et al., 2024; Xu et al., 2025a). Owing to the scaling inference compute of long-CoT reasoning, large reasoning models (LRMs) can unleash their reasoning capabilities and achieve better performance in various reasoning tasks (Shao et al., 2024; Chen et al., 2024b). However, the performance of LRMs highly relies on high-quality intermediate reasoning trajectories (Yang et al., 2025c), which are usually costly and time-consuming to obtain (Peng et al., 2025). In response to this issue, the “self-improvement” paradigm has emerged, *i.e.*, models iteratively improve themselves by using the self-generated reasoning trajectories as training data, thereby reducing their dependence on external supervision.

In the context of LRMs, several self-improvement approaches have been proposed (Zelikman et al., 2022; Singh et al., 2023), which boost LRMs’ performance on conventional reasoning tasks effectively. However, in our preliminary experiments (Figure 1), we found that they often fall short in complex reasoning tasks, *e.g.*, medical question-answering (Huang et al., 2025b). More seriously, they might suffer from *model collapse*, where models’ performance degrades due to iterative self-training on model-generated data (Bertrand et al., 2024). Through a series of analyses (§2.2), we reveal that these methods have two major shortcomings: **① data imbalance**, *i.e.*, most training samples are simple, whereas challenging yet crucial samples are scarce; **② overthinking**, *i.e.*, many undesired solutions with redundant and repetitive reasoning steps are used for self-training. Several prior studies also recognize these shortcomings and

attempt to address them. An intuitive way to mitigate data imbalance is to allocate more trials to difficult queries (Tong et al., 2024; Ding et al., 2025). Although effective, it relies on increased inference overhead to obtain new solutions, while overlooking the potential value of prior failed attempts. On the other hand, to alleviate overthinking, the key lies in quantifying the reasoning redundancy. Most existing works achieve this by designing length-oriented metrics (Team et al., 2025; Munkhbat et al., 2025). While achieving remarkable efficiency, overly emphasizing length reduction may hinder models’ deep reasoning and lead to performance degradation (Dai et al., 2025b). Thus, there arises a question: *can we explore a more effective self-improvement training method to make LRM both better and faster?*

To achieve this goal, we propose **HSIR**, which effectively **Harnesses Self-Improvement** in large **R**easoning models via two simple-yet-effective approaches. First, to collect more correct solutions for difficult queries, HSIR introduces a **verify-then-exit** (*VeriExit*) sampling strategy, which verifies the correctness of intermediate reasoning steps in prior failed solutions and self-truncates the reasoning once the current step arrives at the ground-truth answer. The motivation of *VeriExit* is that, within the failed solution, LRMs may have arrived at the correct answer during intermediate reasoning, yet ultimately failed to produce the accurate outcome due to reasoning deviation. Second, motivated by the intuition that a high similarity among intermediate reasoning steps often signals redundant or repetitive thinking, HSIR designs an **Intrinsic Diversity** score (denoted as *InDiv*) to quantify overthinking using the internal states of LRMs. In practice, *InDiv* performs an attention-aware eigenvalue analysis on the hidden representations of intermediate reasoning steps, where those with smaller eigenvalues are repetitive. Overall, by efficiently collecting more correct solutions and filtering out undesired overthinking solutions, HSIR can ensure the diversity and conciseness of training data, thus achieving better reasoning performance and efficiency.

We apply our HSIR to two iterative post-training paradigms: supervised fine-tuning (SFT) and preference learning. Extensive results on seven LLMs and five reasoning tasks show that our HSIR not only outperforms the other counterparts by a clear margin, but also effectively improves the reasoning efficiency. Furthermore, we expand our methods to the Reinforcement Learning from Verifiable Rewards (RLVR) training paradigm, and propose **H-GRPO**, an enhanced GRPO (Shao et al., 2024) algorithm that leverages the *InDiv* scores as an external reward to smoothly alleviate the overthinking. More comparative results prove the superiority of H-GRPO. Additionally, more in-depth analyses prove the effectiveness of HSIR’s important components, and indicate that HSIR brings better model generalization. To summarize, our contributions are three-fold: (1) We reveal two shortcomings of self-improvement in LRMs, and

propose HSIR that leverages two simple-yet-effective approaches to alleviate them. (2) HSIR can be adopted to various post-training paradigms. Among which, we further expand it to the RLVR training paradigm, and propose an enhanced GRPO algorithm, H-GRPO. (3) Extensive results show that HSIR can consistently and significantly improve the reasoning performance and efficiency for various LLMs.

2. Rethinking Self-improvement in LRMs

2.1. Preliminaries

Considering that we have a base LLM $\mathcal{M}_{base}$, a small seed dataset $\mathcal{S} = \{(x_i, r_i, y_i)\}_{i=1}^N$ and a larger answer-labeled but reasoning-trace-free dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^M$ ($M \gg N$), where x_i is the query, $r_i = [r_{i,1}, \dots, r_{i,L}]$ is the corresponding reasoning trajectory with L intermediate steps, and y_i is the ground-truth answer. We first fine-tune $\mathcal{M}_{base}$ on $\mathcal{S}$ to make it have basic long-CoT reasoning ability, and denote the tuned model as $\mathcal{M}_0$. The goal of self-improvement is to enhance the reasoning performance of $\mathcal{M}_0$ by iteratively self-training using its own solutions on $\mathcal{D}$ over T cycles. Specifically, let $\mathcal{M}_t$ denote the model at the t -th iteration ($t \in [1, T]$), the self-improvement training process generally involves the following steps:

Self-generation. At t -th iteration, for each query $x_i \in \mathcal{D}$, we enforce $\mathcal{M}_{t-1}$ to generate multiple solutions $\{(\hat{r}_i^k, \hat{y}_i^k)\}_{k=1}^K$, where $k \in [1, K]$ and K denotes the sampling time for each query. By doing so, we can obtain the self-generated dataset $\hat{\mathcal{D}}_t = \{(x_i, \hat{r}_i^k, \hat{y}_i^k) \mid k \in [1, K]\}$.

Self-training. The self-training process differs across various post-training paradigms. During **SFT** training, the ground-truth answer y_i is used to verify the correctness of candidate solutions $\{(r_i^k, \hat{y}_i^k)\}_{k=1}^K$, where only correct solutions with $\mathbb{I}(\hat{y}_i, y_i) = 1$ are filtered to form the pseudo-labeled dataset $\hat{\mathcal{D}}_t^{correct} = \{(x_i, \hat{r}_i^k, \hat{y}_i^k) \mid \mathbb{I}(\hat{y}_i^k, y_i) = 1\}$. Notably, to alleviate the model collapse problem, we follow the prior studies (Alemohammad et al., 2024) and use the combination of the original seed $\mathcal{S}$ and the pseudo-labeled $\hat{\mathcal{D}}_t^{correct}$ as the final training dataset $\mathcal{D}_t = \mathcal{S} \cup \hat{\mathcal{D}}_t^{correct}$. Considering that continually fine-tuning $\mathcal{M}_{t-1}$ would lead to overfitting, we fine-tune $\mathcal{M}_{base}$ on $\mathcal{D}_t$ to obtain the new $\mathcal{M}_t$, following previous practice (Zelikman et al., 2022). In particular, we optimize $\mathcal{M}_t$ using the standard negative log likelihood (NLL) loss function:

$$\mathcal{L}_{\text{SFT}} = \mathbb{E}_{\mathcal{D}_t} \left[-\log \frac{\mathcal{M}_\theta(\hat{r}_i^k, \hat{y}_i^k | x_i)}{|\hat{r}_i^k| + |\hat{y}_i^k|} \right], \quad (1)$$

where $\mathcal{M}_\theta$ initialized with $\mathcal{M}_{base}$ denotes the current tuned model that will become next model $\mathcal{M}_t$. For the implementation of **preference learning**, we utilize the Direct Preference Optimization (DPO) (Rafailov et al., 2023). First, for each query $x_i \in \hat{\mathcal{D}}_t$, we split the candidate solutions

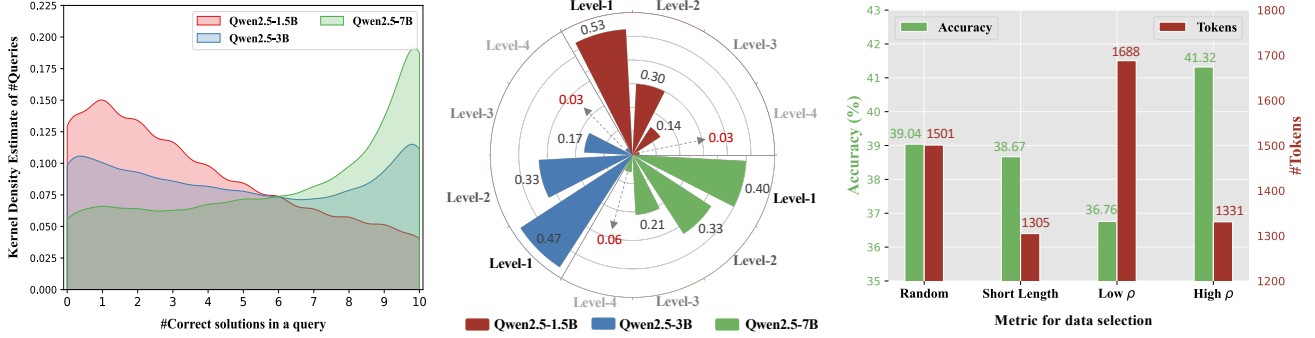


Figure 2. **Left:** Distribution of the number of correct solutions in a single query. **Middle:** Distribution of self-generated training samples with different difficulty levels, where level-1 means the simplest and level-4 means the most difficult. **Right:** Performance comparison between tuned Qwen2.5-1.5B models using different data selection methods. Here, all experiments are based on the MedQA task.

into two sets: winner $\{(x_i, \hat{r}_i^{k_w}, \hat{y}_i^{k_w}) \mid \mathbb{I}(\hat{y}_i^{k_w}, y_i) = 1\}$ and loser $\{(x_i, \hat{r}_i^{k_l}, \hat{y}_i^{k_l}) \mid \mathbb{I}(\hat{y}_i^{k_l}, y_i) = 0\}$. Then, each winning solution and a randomly-selected losing solution are paired to construct the preference set $\mathcal{D}_t^{\text{pairs}} = \{(x_i, \hat{r}_i^{k_w}, \hat{y}_i^{k_w}), (x_i, \hat{r}_i^{k_l}, \hat{y}_i^{k_l}) \mid k_w, k_l \in [1, K]\}$. Lastly, we can obtain the $\mathcal{M}_t$ by continually optimizing $\mathcal{M}_{t-1}$ on $\mathcal{D}_t^{\text{pairs}}$. Inspired by Pang et al. (2024), we employ an enhanced DPO algorithm that combines the standard DPO loss function and NLL loss function on winning solutions to ensure the training stability, which is formulated as follows:

$$\begin{aligned} \mathcal{L}_{\text{DPO+NLL}} &= \mathcal{L}_{\text{DPO}} + \alpha_{\text{nll}} \cdot \mathcal{L}_{\text{NLL}}(\hat{r}_i^{k_w}, \hat{y}_i^{k_w} \mid x_i) \\ &= \mathbb{E}_{\mathcal{D}_t^{\text{pairs}}} \left[-\log \sigma \left(f(\hat{r}_i^{k_w}, \hat{y}_i^{k_w} \mid x_i) - f(\hat{r}_i^{k_l}, \hat{y}_i^{k_l} \mid x_i) \right) \right. \\ &\quad \left. - \alpha_{\text{nll}} \cdot \frac{\log \mathcal{M}_\theta(\hat{r}_i^{k_w}, \hat{y}_i^{k_w} \mid x_i)}{|\hat{r}_i^{k_w}| + |\hat{y}_i^{k_w}|} \right], \end{aligned} \quad (2)$$

where σ is the sigmoid function, $f(\cdot \mid x_i) = \beta \log \frac{\mathcal{M}_\theta(\cdot \mid x_i)}{\mathcal{M}_{t-1}(\cdot \mid x_i)}$, $\mathcal{M}_\theta$ is the policy model initialized with $\mathcal{M}_{t-1}$, α_{nll} and β are coefficients that are empirically set to 0.5 and 0.1.

2.2. Empirical Analyses

Settings. We conduct preliminary experiments by fine-tuning Qwen2.5-1.5B/3B/7B instruct models (Yang et al., 2024) on a challenging medical reasoning dataset, *i.e.*, MedQA (Jin et al., 2021). Since the original MedQA does not contain any reasoning traces, we prompt the proprietary DeepSeek-R1 to generate the seed reasoning data. By filtering out incorrect solutions, we ultimately obtained a new training set containing 9.3K reasoning samples. From this set, we randomly selected 1K as seed data $\mathcal{S}$, while treating the remaining samples (without using their reasoning traces) as unlabeled data $\mathcal{D}$. For self-improvement training, the total sampling time K is set to 10, and the number of iterations T is set to 1 for faster experiment validation.

Findings. Through extensive analyses on the self-generated training samples, we empirically found that:

❶ **Data Imbalance:** Figure 2 (Left) illustrates the distribution of the number of correct solutions in a query. As seen, there is a positive correlation between the number of correct solutions and model capabilities, where stronger models (*e.g.*, Qwen2.5-7B) can collect more accurate solutions. However, for the difficult queries that are proven to be more crucial for further training (Liu et al., 2024), these models still struggle to collect sufficient correct solutions, thus leading to data imbalance. Specifically, although for the powerful Qwen2.5-7B, there are more than 500 queries that did not obtain any correct solutions. To have a close look, based on the number of correct solutions in a query, we evenly split the queries into four levels. Figure 2 (Middle) shows the distribution of self-generated data at different levels, indicating that **most training samples are simple, whereas challenging yet crucial samples are scarce.**

❷ **Overthinking:** To verify the effect of data quality for self-training, we first introduce a **reasoning diversity metric** to measure the reasoning diversity. Specifically, as shown in Algorithm 1, for each reasoning trajectory $\hat{r}_i^k$, we first convert all reasoning steps into sentence embeddings using the BGE-m3 model (Chen et al., 2024c), and then calculate the cosine distance between each step and its nearest neighbor in the current subset. The reasoning steps with cosine distance below the threshold τ_{sim} are filtered out. The τ_{sim} is empirically set to 0.85 in our work. The metric ρ is defined as the ratio of unfiltered steps to all steps:

$$\rho_i^k = \frac{|\mathcal{U}(\hat{r}_i^k)|}{|\hat{r}_{i,1}^k, \dots, \hat{r}_{i,L}^k|}, \quad \rho_i^k \in (0, 1] \quad (3)$$

where $\mathcal{U}(\cdot)$ denotes the set of unfiltered steps. Based on this metric, we conduct comparative experiments by using the solution with the highest and lowest ρ_i^k for each query as self-training data. For reference, we also employ a random and a length-penalty method as baselines, *i.e.*, using a randomly selected solution and the shortest solution for self-training, respectively. Figure 2 (Right) shows the comparative results of Qwen2.5-1.5B models on MedQA,

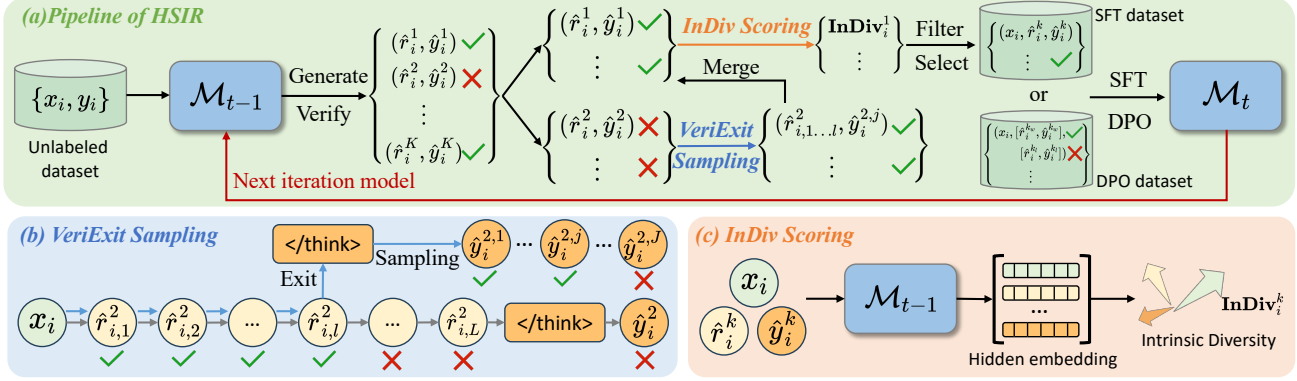


Figure 3. (a) Pipeline of self-improvement training with HSIR. After generating candidate solutions for each query, we first employ our (b) *VeriExit* sampling strategy to collect more accurate solutions for difficult queries, and then quantify the overthinking of correct solutions via our (c) *InDiv* metric. Lastly, the accurate, diverse, and concise solutions are selected for iterative SFT/DPO training.

using the average accuracy and number of output tokens as metrics. As seen, compared to using random solutions, self-training on solutions with low ρ scores indeed results in more inference overhead and lower accuracy, while using the solutions with high ρ scores can effectively alleviate this problem. Notably, although self-training on shorter solutions is also beneficial to improve reasoning efficiency, it would lead to performance degradation. These results suggest that **self-training with redundant and repetitive reasoning steps undermines the accuracy and conciseness of models’ reasoning**, *i.e.*, leading to overthinking.

3. Methodology

Motivation and Intuition of HSIR To alleviate the above problems, we propose HSIR that harnesses self-improvement in LRMs via two novel approaches. First, to address data imbalance, we build upon the insight (Yang et al., 2025b; Dai et al., 2025b) that *failed solutions are not entirely incorrect but often contain valuable, partially correct reasoning steps before deviating*. Instead of discarding these outputs, we introduce *VeriExit*, a novel **trajectory recycling** strategy. It efficiently salvages valid initial reasoning from failed attempts to generate correct solutions for difficult queries. This approach provides a significant efficiency gain over costly resampling from scratch. Second, to combat overthinking, our empirical analysis in §2.2 validates that reasoning diversity is a potent signal, yet the preliminary metric (Eq. 3) relies on costly external models. We propose a more elegant solution by harnessing the dense semantic information already present in a model’s internal states. We introduce the **Intrinsic Diversity (*InDiv*) score**, an efficient metric that measures diversity directly from the models’ hidden representations. This makes *InDiv* an **entirely intrinsic** measure that eliminates external dependencies and can be computed with minimal to zero overhead. Figure 3 illustrates the overview of our HSIR.

Verify-then-Exit Sampling. We first select the failed solutions from $\hat{\mathcal{D}}_t$ to obtain a new set $\hat{\mathcal{D}}_t^{wrong} = \{(x_i, \hat{r}_i^k, \hat{g}_i^k) \mid \mathbb{I}(\hat{g}_i^k, y_i) = 0\}$. For each $\hat{r}_i^k \in \hat{\mathcal{D}}_t^{wrong}$, we verify the correctness of intermediate reasoning steps $[\hat{r}_{i,1}^k, \dots, \hat{r}_{i,L}^k]$ by determining whether the $\hat{r}_{i,l}^k$ ($l \in [1, L]$) arrives at the ground-truth answer y_i , *e.g.*, explicitly mentioning “answer is $\{y_i\}$ ”¹. Once $\hat{r}_{i,l}^k$ arrives at y_i , we truncate the subsequent reasoning steps and insert the exit prompt “\n\n</think>\n<answer>\n” at the truncated position. The obtained new query is then fed into $\mathcal{M}_{t-1}$ to stop further reasoning and sample the answers for J times ($J < K$), where the successful ones are merged into $\hat{\mathcal{D}}_t^{correct}$. If the total number of candidate solutions of each x_i exceeds K , we randomly sample K solutions to maintain consistency between the training budgets of our method and those of vanilla self-improvement methods. Notably, for the “absolutely hard queries” where no valid intermediate steps exist, *VeriExit* silently skips them (producing no new samples), gracefully degrading to the same behavior as the standard self-improvement data collection method. Thus, such a sampling process will not lead to much inference overhead, compared to resampling from scratch.

Intrinsic Diversity Score. To quantify overthinking, we leverage the LLMs’ internal states to measure the semantic diversity of reasoning steps. Specifically, for each correct solution $(x_i, \hat{r}_i^k, \hat{g}_i^k)$, we obtain its hidden representation $\mathbf{H}_i^k \in \mathbb{R}^{d \times m}$ at the middle layer of $\mathcal{M}_{t-1}$, where d is the dimension of hidden states and m is the number of all tokens in the solution. We choose the middle layer as it encodes richer and more useful semantic information (Skean et al., 2025; Azaria & Mitchell, 2023). Inspired by the fact that eigenvalues of the covariance matrix can capture the divergence and correlation between different embeddings (Chen et al., 2024a), we calculate the eigenvalues of the cross-

¹We adopt a text-matching approach by default for efficiency, and analyses of more methods can be found in Appendix D.1.

covariance for $\mathbf{H}_i^k$ to measure the reasoning diversity:

$$\begin{aligned}\Sigma_i^k &= \mathbf{H}_i^k{}^\top \cdot \mathbf{J}_d \cdot \mathbf{H}_i^k, \\ \text{Eig}_i^k &= \frac{1}{m} \log \det(\Sigma_i^k) = \frac{1}{m} \sum_{u=1}^m \log(\lambda_{i,u}^k),\end{aligned}\quad (4)$$

where $\mathbf{J}_d = \mathbf{I}_d - \frac{1}{d} \mathbf{1}_{d \times d}$ is the centering matrix, $\mathbf{I}_d \in \mathbb{R}^d$ is the identity matrix, $\mathbf{1}_{d \times d} \in \mathbb{R}^{d \times d}$ is the all-one matrix, $\det(\cdot)$ means the determinant of matrix, and $\{\lambda_{i,u}^k\}_{u=1}^m$ denotes the singular values of matrix Σ_i^k . Furthermore, considering that some important tokens with higher attention weights might contribute more to the reasoning process, we enhance the above method via an attention-aware weighting mechanism. In practice, let $\text{Atten}_{i,u}^k \in (0, 1)$ denote the normalized maximum self-attention weight for u -th token ($u \in [1, m]$) among all self-attention heads, we can obtain our final reasoning diversity metric, denoted as **Intrinsic Diversity** score (*InDiv* in short) to distinguish it from Eq. 3:

$$\begin{aligned}\text{Score}_i^k &= \sum_{u=1}^m \left[\text{Atten}_{i,u}^k \cdot \log(\lambda_{i,u}^k) \right]; \sum_{u=1}^m \text{Atten}_{i,u}^k = 1 \\ \text{InDiv}_i^k &= \frac{\text{Score}_i^k - \text{mean}(\text{Score}_i)}{\text{std}(\text{Score}_i)}.\end{aligned}\quad (5)$$

When the reasoning steps are repetitive and have similar semantics, the hidden representations will be highly correlated, and their semantic entropy and *InDiv* scores will be small (more analyses are in Appendix D.2). After calculating the *InDiv* scores of all candidate correct solutions for x_i , we filter the undesired ones with scores below the threshold τ . Finally, we can obtain more accurate, diverse, and concise reasoning data for effective self-training. The pseudo-code of HSIR is shown in Algorithm 2, and the efficiency analysis of HSIR can be found in Appendix D.9.

4. Experiments

4.1. Experimental Setup

Tasks and Datasets. We mainly assess the effectiveness of HSIR on medical reasoning and mathematical reasoning tasks, using the MedQA (Jin et al., 2021) and GSM8K (Cobbe et al., 2021) datasets, respectively. Moreover, to verify the universality of HSIR, we additionally conduct experiments on several popular reasoning tasks, *i.e.*, AI2 Reasoning Challenge’s challenge set (ARC) (Clark et al., 2018) for scientific reasoning, CommonsenseQA (CSQA) (Talmor et al., 2019) for commonsense reasoning, and StrategyQA (StraQA) (Geva et al., 2021) for multi-hop reasoning. For each task, we follow the settings in §2.2 and use the DeepSeek-R1 to obtain the seed reasoning data. Some training samples are provided in Table 9 and 10. For evaluation, we report the zero-shot results on the original test sets of each benchmark using the average accuracy and

number of generated tokens as metrics, where the latter metric can reflect the wall-clock inference latency.

Training Details. We conduct main experiments using Qwen2.5-1.5B/3B/7B (Yang et al., 2024), Qwen3-1.7B (Yang et al., 2025a), Phi-3.5-mini (Abdin et al., 2024), Mistral-7B (Jiang et al., 2023), and LLaMA3-8B (Dubey et al., 2024) instruct models. During the implementation of HSIR, the sampling times K and J are set to 10 and 5, respectively. The sampling temperature is 1.0, and the maximum output length is 2,048. The filter threshold τ is set to -0.5. Notably, for the post-training of Qwen2.5 models, the self-improvement iteration T is set to 3, but for the other LLMs, it is set to 1 due to limited computational resources. During inference, we use greedy decoding with a temperature of 0 for reproducibility. The maximum output length is set to 4,096. More data and training details are shown in Appendix C.

Baselines. To verify the superiority of our HSIR, we compare it with various training-based baseline methods:

- **SFT-Initial:** Standard fine-tuning $\mathcal{M}_{base}$ on the seed data $\mathcal{S}$ to obtain the initial SFT model $\mathcal{M}_0$.
- **SFT-Oracle:** Standard fine-tuning $\mathcal{M}_{base}$ on the combination of $\mathcal{S}$ and $\mathcal{D}$ with ground-truth reasoning data, which can be regarded as an upper-bound of SFT training.
- **STaR** (Zelikman et al., 2022): Sampling a solution using greedy decoding for each query $x_i \in \mathcal{D}$, where the correct solutions are used to iteratively fine-tune the models.
- **ReST^{EM}** (Singh et al., 2023): Extending STaR by sampling K solutions for each query $x_i \in \mathcal{D}$, where all correct solutions are used for iterative SFT training.
- **RFT** (Yuan et al., 2023): Similar to ReST^{EM} but not iterative. To maintain consistent training budgets, we sample $T \times K$ candidate solutions for each query $x_i \in \mathcal{D}$.
- **ReGenesis** (Peng et al., 2025): Prompting $\mathcal{M}_{base}$ to self-synthesize reasoning paths by converting general reasoning guidelines into task-specific ones, which are used for once self-training. Since it is not designed for long-CoT reasoning, we do not compare its reasoning efficiency.
- **IRPO** (Pang et al., 2024): Sampling K solutions for each query, where correct and incorrect solutions are paired to construct the preference data for iterative DPO training.

For all baselines, we keep a fixed data synthesis budget. Moreover, since our goal is to propose a self-improvement training method, we do not compare HSIR with inference-time methods in the main experiments. More comparisons with inference-time methods are shown in Appendix D.8.

Table 1. Performance comparison between Qwen2.5 models using different training methods on MedQA and GSM8K. “|Train|” denotes the average number of training samples among all models and tasks. “Overall” denotes the average accuracy and number of output tokens.

Methods	Train	Qwen2.5-1.5B		Qwen2.5-3B		Qwen2.5-7B		Overall	
		MedQA	GSM8K	MedQA	GSM8K	MedQA	GSM8K	Accuracy	Tokens
SFT-Initial	1.0K	38.10	63.99	49.02	77.18	62.45	83.93	62.45	1,536
SFT-Oracle	8.1K	46.58	71.57	58.68	84.31	73.99	87.79	70.49 $\uparrow 8.04$	1,392 $\downarrow 9.4\%$
(a) Iterative Self-improvement SFT Training									
RFT	132.2K	42.42	71.19	54.60	83.55	64.89	87.87	67.42 $\uparrow 4.97$	1292 $\downarrow 15.9\%$
ReGenesis	127.2K	44.46	66.26	50.67	79.91	62.22	89.99	65.59 $\uparrow 3.13$	363
STaR									
Iteration 1	5.7K	35.98	69.75	49.25	81.96	61.19	87.72	64.31 $\uparrow 1.86$	1,379 $\downarrow 10.2\%$
Iteration 2	6.1K	37.78	70.96	50.82	81.27	61.04	88.48	65.06 $\uparrow 2.61$	1,328 $\downarrow 13.5\%$
Iteration 3	6.4K	38.49	72.02	47.76	81.58	61.59	87.49	64.82 $\uparrow 2.37$	1,288 $\downarrow 16.1\%$
ReST ^{EM}									
Iteration 1	44.7K	41.63	69.75	55.22	83.95	64.18	88.17	67.15 $\uparrow 4.70$	1,268 $\downarrow 17.5\%$
Iteration 2	51.0K	42.81	74.45	56.25	85.13	65.28	90.22	69.02 $\uparrow 6.57$	1,160 $\downarrow 24.5\%$
Iteration 3	53.6K	45.48	75.13	56.48	86.04	65.28	89.99	69.73 $\uparrow 7.28$	1,114 $\downarrow 27.5\%$
HSIR-SFT (Ours)									
Iteration 1	33.6K	45.33	71.72	55.70	86.13	67.32	88.78	69.16 $\uparrow 6.71$	1,075 $\downarrow 30.0\%$
Iteration 2	36.6K	46.50	76.04	56.32	86.51	67.87	90.83	70.68 $\uparrow 8.23$	950 $\downarrow 38.1\%$
Iteration 3	38.8K	46.58	76.88	57.58	86.81	68.74	91.36	71.33$\uparrow 8.88$	896$\downarrow 41.7\%$
(b) Iterative Self-improvement DPO Training									
IRPO									
Iteration 1	22.7K	39.04	70.51	47.99	85.37	64.26	91.43	66.43 $\uparrow 3.98$	1,359 $\downarrow 11.5\%$
Iteration 2	27.5K	41.08	75.51	49.33	86.66	63.24	91.95	67.96 $\uparrow 5.51$	1,294 $\downarrow 15.7\%$
Iteration 3	20.5K	43.91	75.36	49.10	87.19	60.57	91.87	68.00 $\uparrow 5.55$	1,271 $\downarrow 17.2\%$
HSIR-DPO (Ours)									
Iteration 1	17.9K	48.31	75.36	55.77	87.04	67.32	91.36	70.86 $\uparrow 8.41$	1,007 $\downarrow 34.4\%$
Iteration 2	23.6K	49.10	76.65	59.15	87.49	68.58	91.96	72.16 $\uparrow 9.70$	921 $\downarrow 40.0\%$
Iteration 3	19.6K	50.90	78.09	60.64	87.53	70.46	92.49	73.35$\uparrow 10.90$	885$\downarrow 42.4\%$

4.2. Main Results

HSIR outperforms the other baseline methods across all post-training settings. Table 1 reports the comparative results (%) of Qwen2.5 family models. As seen, self-improvement training on the relatively simple GSM8K task performs better against the challenging MedQA task, confirming that self-improvement methods fall short in complex reasoning tasks. More specifically, during SFT on MedQA, STaR struggles to enhance the LRMs’ reasoning performance, and even leads to performance degradation, *e.g.*, from 38.10% to 35.98% in Qwen2.5-1.5B. By sampling more diverse solutions, RFT and ReST^{EM} alleviate this problem, indicating the importance of self-training with diverse reasoning data. While in the DPO phase, the effectiveness of self-improvement is more dependent on the quality of self-generated data, as preference learning is more data-sensitive. By selecting more diverse and concise self-generated data for training, our HSIR can effectively unleash the reasoning capabilities of LRMs, thus achieving better performance against the other baselines. For instance, our HSIR-DPO outperforms the IRPO by a clear margin, *i.e.*, bringing +5.35% average performance gains.

HSIR effectively improves the reasoning performance and efficiency of LRMs in both tasks. As aforementioned, we also evaluate the reasoning efficiency of LRMs by measuring the number of output tokens. From Table 1, it can be seen that nearly all self-improvement methods reduce the average output tokens. We conjecture that models can sometimes generate concise reasoning paths, which helps guide the efficient reasoning of models. Nevertheless, as shown in our preliminary analysis (§2.2), self-training with overthinking solutions would damage this effect and lead to suboptimal reasoning efficiency. Owing to our *InDiv* metric, we can filter these overthinking solutions and effectively improve reasoning efficiency by reducing up to 42.4% output tokens. These results confirm the significance of alleviating overthinking and prove the effectiveness of HSIR.

HSIR brings consistent and significant performance gains among all model sizes and types. Table 2 presents the results of other LRMs. Notably, due to limited computation resources, we only perform the SFT training using STaR, ReST^{EM}, and our HSIR for one iteration. As seen, HSIR continues to outperform the other baseline methods across all models. Specifically, in LLaMA3-8B, compared

Table 2. Performance comparison between the other models using different self-improvement SFT methods on MedQA and GSM8K.

Methods	Qwen3-1.7B		Phi-3.5-mini		Mistral-7B		LLaMA3-8B		Overall	
	MedQA	GSM8K	MedQA	GSM8K	MedQA	GSM8K	MedQA	GSM8K	Accuracy	Tokens
SFT-Initial	51.61	87.03	66.14	82.56	55.93	64.59	64.57	79.98	69.05	1,540
SFT-Oracle	55.77	87.72	74.07	87.95	70.15	79.08	73.76	86.73	76.90 $\uparrow 7.85$	1,383 $\downarrow 10.2\%$
STaR	52.87	86.96	65.99	86.28	53.57	68.54	59.63	80.59	69.30 $\uparrow 0.25$	1,442 $\downarrow 6.3\%$
ReST ^{EM}	54.67	88.61	67.64	87.86	58.13	72.71	67.79	81.20	72.33 $\uparrow 3.28$	1,365 $\downarrow 11.4\%$
HSIR-SFT	55.30	89.16	71.17	88.32	61.27	75.82	69.68	86.66	74.67$\uparrow 5.62$	1,195$\downarrow 22.4\%$

Table 3. Ablation study on VeriExit and InDiv approaches.

Method	MedQA		GSM8K	
	Accuracy	Tokens	Accuracy	Tokens
SFT-Initial	38.10	1,779	63.99	1,666
STaR	35.98	1,651	69.75	1,377
ReST ^{EM}	41.63	1,424	69.75	1,382
HSIR-SFT (All)	45.33	1,064	71.72	1,181
(a) Analysis of sampling strategy (without data filtering)				
-w/ Answer-driven	41.63	1,421	69.29	1,433
-w/ VeriExit (Ours)	44.46	1,293	70.74	1,361
(b) Analysis of overthinking metric (without extra sampling)				
-w/ Length-driven	42.36	1,321	70.66	1,266
-w/ InDiv (Ours)	42.66	1,260	71.34	1,256

to powerful ReST^{EM}, HSIR achieves +1.89% and +5.46% performance gains for MedQA and GSM8K, respectively. Overall, HSIR brings +5.62% average performance gains and reduces 22.4% output tokens against the initial SFT models, showing its universality and superiority.

4.3. More Analyses

Ablation Study. In this part, we validate the important components of HSIR, *i.e.*, VeriExit sampling strategy and InDiv metric. Firstly, for the analysis of sampling strategy, we ignore the overthinking metric and do not perform the data filtering. To verify the effectiveness of VeriExit, we compare it with “-w Answer-driven” that uses the ground-truth answer to guide models’ reasoning for collecting more correct solutions (Ding et al., 2025). Similarly, for the analysis of overthinking metric, we do not use extra sampling methods, and compare our InDiv with “-w Length-driven” that leverages the length of solutions as the metric. In practice, the candidate solutions with $\frac{\text{len}(\hat{r}_i^k) - \text{mean}(\text{len}(\hat{r}_i))}{\text{std}(\text{len}(\hat{r}_i))}$ are filtered, where $\text{len}(\hat{r}_i^k)$ denotes the length of $\hat{r}_i^k$ and τ_{len} is the length threshold set to 0.5 in this experiment. Table 3 reports the results of Qwen2.5-1.5B models after one iteration of self-improvement SFT training. Compared to the full HSIR, removing VeriExit or InDiv results in performance degradation, indicating their effectiveness. Moreover, our proposed methods consistently perform better than their counterparts. For instance, VeriExit outperforms the “-w Answer-driven”

Table 4. Comparison results on more reasoning benchmarks. We perform the self-improvement SFT training for one iteration.

Method	Qwen2.5-1.5B			Overall	
	ARC	StraQA	CSQA	Accuracy	Tokens
SFT-Initial	68.33	59.10	60.03	62.49	1,447
SFT-Oracle	69.38	60.84	74.20	68.14 $\uparrow 5.65$	1,333 $\downarrow 7.9\%$
STaR	68.58	61.28	62.74	64.20 $\uparrow 1.71$	1,357 $\downarrow 6.2\%$
ReST ^{EM}	69.78	62.30	77.64	66.74 $\uparrow 4.25$	1,254 $\downarrow 13.3\%$
HSIR-SFT	72.45	64.63	72.48	69.85$\uparrow 7.36$	1,043$\downarrow 27.9\%$

by 2.14% average performance gains. These comparative results demonstrate the superiority of VeriExit and InDiv.

Evaluation on more reasoning benchmarks. To verify the generality of our HSIR, we additionally evaluate it on more reasoning tasks, including ARC (Clark et al., 2018) for scientific reasoning, CSQA (Talmor et al., 2019) for commonsense reasoning, and StraQA (Geva et al., 2021) for multi-hop reasoning. Table 4 shows the comparative results of Qwen2.5-1.5B models. Notably, we perform the self-improvement SFT training for one iteration. As seen, our HSIR-SFT consistently outperforms the other counterparts by a clear margin. More specifically, compared to the base model, HSIR-SFT brings +7.36% average performance gains and reduces 27.9% average inference tokens. These results can prove the generality of our method.

Expand to GRPO Training. In addition to SFT and DPO training, reinforcement learning from verifiable rewards via the GRPO (Shao et al., 2024) algorithm is also a popular and effective way to enhance LLMs’ reasoning performance. Instead of explicitly supervising the reasoning trajectory, GRPO enables LLMs to learn from free exploration via outcome rewards, *e.g.*, binary accuracy reward. Although effective, GRPO training also suffers from the overthinking problem. To this end, we propose to improve the GRPO by leveraging our InDiv score as an extra reward, and denote this method as **H-GRPO**. The implementation details of H-GRPO can be found in Appendix B. Intuitively, by encouraging LLMs to generate diverse and concise reasoning paths, H-GRPO can effectively alleviate overthinking and result in better performance. It is noteworthy that the calculation of InDiv scores is fast and would not lead to much training

Table 5. Performance comparison between Qwen2.5 models using different GRPO algorithms.

Methods	Qwen2.5-1.5B		Qwen2.5-3B		Qwen2.5-7B		Overall	
	MedQA	GSM8K	MedQA	GSM8K	MedQA	GSM8K	Accuracy	Tokens
GRPO	46.34	73.08	57.03	82.49	66.46	89.61	69.17	974
+ Long2Short	46.50	72.63	58.21	82.26	66.14	91.13	69.48 $\uparrow$ 0.31	668 $\downarrow$ 31.4%
+ CosFn	47.96	71.72	55.22	82.93	63.71	89.31	68.48 $\downarrow$ 0.69	966 $\downarrow$ 0.86%
H-GRPO (Ours)	48.15	74.60	58.98	83.09	68.03	91.43	70.71 $\uparrow$ 1.54	710 $\downarrow$ 27.12%

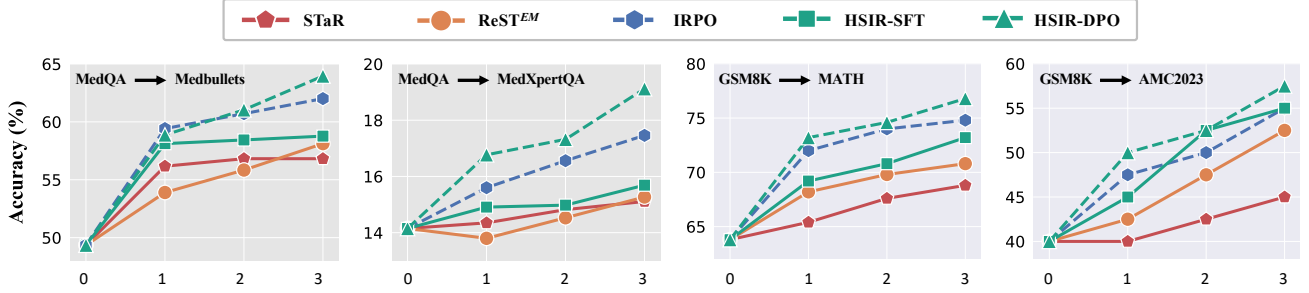


Figure 4. Comparison of OOD results between Qwen2.5-7B models trained with different iterative self-improvement post-training methods. The x-axis denotes the index of self-improvement training iteration.

latency. We apply our H-GRPO to reinforce the $\mathcal{M}_0$ models using the $\mathcal{D}$ dataset, and report the results of Qwen2.5 family models in Table 5. Notably, our goal is not to propose a new state-of-the-art GRPO method, but rather to examine whether our *InDiv* can be incorporated into GRPO to mitigate the overthinking problem. Therefore, we only compare our H-GRPO with two representative GRPO algorithms that also focus on the overthinking problem: *Long2Short* (Team et al., 2025) and *CosFn* (Yang et al., 2025c), which address overthinking by using length-oriented reward functions. As seen, compared to the vanilla GRPO, all improved methods achieve better reasoning efficiency, indicating the validity of extra rewards. However, both length-oriented methods would cause a decrease in reasoning accuracy, e.g., 1.53% average performance drops in Qwen2.5-7B on MedQA. This indicates that overly emphasizing length reduction might hinder LLMs’ deep reasoning and lead to suboptimal results. Conversely, by optimizing the intermediate reasoning process, our H-GRPO can smoothly reduce repetitive and redundant thinking and thus achieve better performance.

Model Generalization. Here, we further investigate the ability of self-improved LLMs to generalize to out-of-distribution (OOD) tasks. Specifically, for models trained on MedQA, we evaluate their performance on two complex medical benchmarks, i.e., Medbullets (4-option) (Chen et al., 2025a) and MedXpertQA (Zuo et al., 2025). While for models trained on GSM8K, we evaluate on the challenging MATH (Hendrycks et al., 2021) and AMC2023 (Mathematical Association of America, 2023). We illustrate the OOD results of Qwen2.5-7B models using different self-training methods in Figure 4. From it, we can observe that: (1)

Table 6. Cross-domain and cross-task OOD evaluation on Qwen2.5-7B under different iterative DPO training methods.

Method	MedQA $\rightarrow$ GPQA	GSM8K $\rightarrow$ MedQA
IRPO		
Iteration 1	23.66	52.63
Iteration 2	22.55 ($\downarrow$ 1.11)	48.39 ($\downarrow$ 4.24)
Iteration 3	22.54 ($\downarrow$ 1.12)	41.32 ($\downarrow$ 11.31)
HSIR-DPO (Ours)		
Iteration 1	28.79	53.49
Iteration 2	31.70 ($\uparrow$ 2.91)	53.57 ($\uparrow$ 0.08)
Iteration 3	34.60 ($\uparrow$ 5.81)	53.47 ($\downarrow$ 0.02)

Compared to iterative SFT, self-improvement with iterative DPO training can generally result in better OOD performance, similar to the finding of Wu et al. (2025). This is consistent with the wisdom that DPO can improve OOD generalization (Kirk et al., 2024). (2) Both HSIR-SFT and HSIR-DPO can achieve consistently better OOD results against the baseline methods. We attribute it to the *VeriExit* sampling strategy in HSIR as it can collect more diverse solutions for generalized self-training. These results confirm our motivation to mitigate the data imbalance problem.

Furthermore, to more rigorously evaluate the OOD generalization of our method, we conduct two additional and more challenging experiments: (1) *Cross-domain*: training on the medical-domain MedQA and testing on the out-of-domain GPQA (Rein et al., 2024); (2) *Cross-task*: training on free-form GSM8K and testing on multiple-choice MedQA. Comparative results on Qwen2.5-7B under iterative DPO training are presented in Table 6. As shown, IRPO suffers from progressive OOD performance degradation

across successive self-improvement iterations, whereas our HSIR not only avoids this decline but even yields consistent OOD improvements (28.79 $\rightarrow$ 34.60 in the cross-domain setting), demonstrating its strong generalization capability.

📌 **Note:** Due to space limitations, we introduce the related works in Appendix A, the implementation and analysis of H-GRPO in Appendix B, more experimental details in Appendix C, and more experiments and analyses in Appendix D. Please refer to the Appendix for more details.

5. Conclusion

In this paper, we reveal and address the limitations of self-improvement post-training in LRMs. Through a series of preliminary analyses, we find that the self-improvement of LRMs usually suffers from data imbalance and overthinking in complex reasoning scenarios. To address these limitations, we propose HSIR, which effectively harnesses self-improvement in LRMs via two simple-yet-effective approaches: *VeriExit* sampling strategy and *InDiv* metric. Specifically, *VeriExit* alleviates the data imbalance by efficiently collecting more correct solutions for difficult queries, while *InDiv* quantifies the overthinking and filters out the undesired candidate solutions. Extensive results show that our HSIR consistently and significantly improves the reasoning performance and efficiency across all model sizes and architectures. Moreover, we also expand our method to the RLVR training paradigm and propose H-GRPO that improves the GRPO by leveraging the *InDiv* scores as an extra reward. Comparative results with two widely-used GRPO variants demonstrate the superiority of our H-GRPO.

Impact Statement

For the ethics of our work, we take ethical considerations very seriously and strictly adhere to the ICML Ethics Policy. This paper proposes a new self-improvement training framework to improve the reasoning performance and efficiency of LRMs. It aims to unleash LRMs’ internal reasoning capabilities, instead of encouraging them to learn privacy knowledge that may cause an ethical problem. Moreover, all base models, training, and evaluation datasets used in this paper are publicly available and have been widely adopted by researchers. Thus, we believe that this research will not pose ethical issues. For the potential societal consequences, we believe that there are many potential societal consequences of our work, but none of which we feel must be specifically highlighted here.

Limitations

Our work has several potential limitations. First, our *InDiv* relies on semantic similarity between adjacent reasoning

steps to detect overthinking. While semantically similar steps often indicate redundant reasoning in common scenarios, this assumption does not always hold. For instance, in rigorous mathematical derivations, consecutive steps may be semantically similar yet logically essential, which could lead *InDiv* to misjudge necessary reasoning as redundant. Second, our *VeriExit* is inherently limited when confronted with “absolutely hard queries” for which no valid intermediate steps exist. Such challenging queries represent problems that lie beyond the model’s current capability boundary, and addressing them remains an important direction for future work, *e.g.*, incorporating stronger external verifiers or curriculum-based training strategies.

Acknowledgements

This research is supported by the RIE2025 Industry Alignment Fund – Industry Collaboration Projects (IAF-ICP) (Award I2301E0026), administered by A*STAR, as well as supported by Alibaba Group and NTU Singapore through Alibaba-NTU Global e-Sustainability CorpLab (ANGEL).

References

- Abdin, M., Aneja, J., Awadalla, H., Awadallah, A., Awan, A. A., Bach, N., Bahree, A., Bakhtiari, A., Bao, J., Behl, H., et al. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2412.08905*, 2024.
- Alemohammad, S., Casco-Rodriguez, J., Luzi, L., Humayun, A. I., Babaei, H., LeJeune, D., Siahkoohi, A., and Baraniuk, R. Self-consuming generative models go mad. In *The Twelfth International Conference on Learning Representations*, 2024.
- Azaria, A. and Mitchell, T. The internal state of an llm knows when it’s lying. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.
- Bertrand, Q., Bose, J., Duplessis, A., Jiralerspong, M., and Gidel, G. On the stability of iterative retraining of generative models on their own data. In *The Twelfth International Conference on Learning Representations*, 2024.
- Cao, Y., Kang, Y., Wang, C., and Sun, L. Instruction mining: Instruction data selection for tuning large language models. In *First Conference on Language Modeling*, 2024.
- Chen, C., Liu, K., Chen, Z., Gu, Y., Wu, Y., Tao, M., Fu, Z., and Ye, J. Inside: LLMs’ internal states retain the power of hallucination detection. In *The Twelfth International Conference on Learning Representations*, 2024a.
- Chen, H., Fang, Z., Singla, Y., and Dredze, M. Benchmarking large language models on answering and explaining

- challenging medical questions. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2025a.
- Chen, J., Cai, Z., Ji, K., Wang, X., Liu, W., Wang, R., Hou, J., and Wang, B. Huatuoqpt-o1, towards medical complex reasoning with llms. *arXiv preprint arXiv:2412.18925*, 2024b.
- Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., and Liu, Z. M3-embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In *Findings of the Association for Computational Linguistics ACL 2024*, 2024c.
- Chen, J. C.-Y., Peng, B. X., Choubey, P. K., Huang, K.-H., Zhang, J., Bansal, M., and Wu, C.-S. Nudging the boundaries of llm reasoning. *arXiv preprint arXiv:2509.25666*, 2025b.
- Chen, Z., Cano, A. H., Romanou, A., Bonnet, A., Matoba, K., Salvi, F., Pagliardini, M., Fan, S., Köpf, A., Mohtashami, A., et al. Meditron-70b: Scaling medical pretraining for large language models. *arXiv preprint arXiv:2311.16079*, 2023.
- Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Dai, M., Liu, S., and Si, Q. Stable reinforcement learning for efficient reasoning. *arXiv preprint arXiv:2505.18086*, 2025a.
- Dai, M., Yang, C., and Si, Q. S-grpo: Early exit via reinforcement learning in reasoning models. *arXiv preprint arXiv:2505.07686*, 2025b.
- Ding, Y., Xi, Z., He, W., Lizhuoyuan, L., Zhai, Y., Xiaowei, S., Cai, X., Gui, T., Zhang, Q., and Huang, X.-J. Mitigating tail narrowing in llm self-improvement via socratic-guided sampling. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2025.
- Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., et al. The llama 3 herd of models. *arXiv e-prints arXiv:2407.21783*, 2024.
- Geva, M., Khashabi, D., Segal, E., Khot, T., Roth, D., and Berant, J. Did aristotle use a laptop? a question answering benchmark with implicit reasoning strategies. *Transactions of the Association for Computational Linguistics*, 2021.
- Giraldo, L. G. S., Rao, M., and Principe, J. C. Measures of entropy from data using infinitely divisible kernels. *IEEE Transactions on Information Theory*, 2014.
- Gulcehre, C., Paine, T. L., Srinivasan, S., Konyushkova, K., Weerts, L., Sharma, A., Siddhant, A., Ahern, A., Wang, M., Gu, C., et al. Reinforced self-training (rest) for language modeling. *arXiv preprint arXiv:2308.08998*, 2023.
- Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring mathematical problem solving with the math dataset. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021.
- Horn, R. A. and Johnson, C. R. *Matrix analysis*. Cambridge university press, 2012.
- Hosseini, A., Yuan, X., Malkin, N., Courville, A., Sordani, A., and Agarwal, R. V-star: Training verifiers for self-taught reasoners. In *First Conference on Language Modeling*, 2024.
- Huang, A., Block, A., Foster, D. J., Rohatgi, D., Zhang, C., Simchowitz, M., Ash, J. T., and Krishnamurthy, A. Self-improvement in language models: The sharpening mechanism. In *The Thirteenth International Conference on Learning Representations*, 2025a.
- Huang, J., Gu, S., Hou, L., Wu, Y., Wang, X., Yu, H., and Han, J. Large language models can self-improve. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
- Huang, Z., Zou, H., Li, X., Liu, Y., Zheng, Y., Chern, E., Xia, S., Qin, Y., Yuan, W., and Liu, P. O1 replication journey—part 2: Surpassing o1-preview through simple distillation, big progress or bitter lesson? *arXiv preprint arXiv:2411.16489*, 2024.
- Huang, Z., Geng, G., Hua, S., Huang, Z., Zou, H., Zhang, S., Liu, P., and Zhang, X. O1 replication journey—part 3: Inference-time scaling for medical reasoning. *arXiv preprint arXiv:2501.06458*, 2025b.

- Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., Casas, D. d. l., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.
- Jin, D., Pan, E., Oufattole, N., Weng, W.-H., Fang, H., and Szolovits, P. What disease does this patient have? a large-scale open domain question answering dataset from medical exams. *Applied Sciences*, 2021.
- Kirk, R., Mediratta, I., Nalmpantis, C., Luketina, J., Hambro, E., Grefenstette, E., and Raileanu, R. Understanding the effects of rlhf on llm generalisation and diversity. In *The Twelfth International Conference on Learning Representations*, 2024.
- Koh, W., Oh, W., Jang, J., Lee, M., Kim, H., Kim, A. Y., Kim, J., Lee, J., Kim, T., and Yun, S.-Y. Adastar: Adaptive data sampling for training self-taught reasoners. In *Proceedings of the 39th International Conference on Neural Information Processing Systems*, 2025.
- Li, G., Al Kader Hammoud, H. A., Itani, H., Khizbullin, D., and Ghanem, B. Camel: communicative agents for "mind" exploration of large language model society. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, 2023.
- Li, X., Ma, Y., Li, C., Bao, K., Yu, Y., Wang, W., Feng, F., and Liu, D. Unified data selection for llm reasoning. 2026.
- Li, Z.-Z., Zhang, D., Zhang, M.-L., Zhang, J., Liu, Z., Yao, Y., Xu, H., Zheng, J., Wang, P.-J., Chen, X., et al. From system 1 to system 2: A survey of reasoning large language models. *arXiv preprint arXiv:2502.17419*, 2025.
- Liu, N. F., Gardner, M., Belinkov, Y., Peters, M. E., and Smith, N. A. Linguistic knowledge and transferability of contextual representations. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2019.
- Liu, W., Zeng, W., He, K., Jiang, Y., and He, J. What makes good data for alignment? a comprehensive study of automatic data selection in instruction tuning. In *The Twelfth International Conference on Learning Representations*, 2024.
- Manakul, P., Liusie, A., and Gales, M. Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, 2023.
- Mathematical Association of America. American mathematics competitions, 2023. URL <https://maa-amc.org/amc/amc-competitions/>.
- Mirzadeh, S. I., Farajtabar, M., Li, A., Levine, N., Matsukawa, A., and Ghasemzadeh, H. Improved knowledge distillation via teacher assistant. In *Proceedings of the AAAI conference on artificial intelligence*, 2020.
- Munkhbat, T., Ho, N., Kim, S. H., Yang, Y., Kim, Y., and Yun, S.-Y. Self-training elicits concise reasoning in large language models. In *Findings of the Association for Computational Linguistics: ACL 2025*, 2025.
- Pang, R. Y., Yuan, W., Cho, K., He, H., Sukhbaatar, S., and Weston, J. Iterative reasoning preference optimization. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, 2024.
- Peng, X., Xia, C., Yang, X., Xiong, C., Wu, C.-S., and Xing, C. Regenes: Llm can grow into reasoning generalists via self-improvement. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Plaat, A., Wong, A., Verberne, S., Broekens, J., van Stein, N., and Bäck, T. Reasoning with large language models, a survey. *arXiv e-prints arXiv:2407.11511*, 2024.
- Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., and Finn, C. Direct preference optimization: your language model is secretly a reward model. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, 2023.
- Rahmath P, H., Srivastava, V., Chaurasia, K., Pacheco, R. G., and Couto, R. S. Early-exit deep neural network-a comprehensive survey. *ACM Computing Surveys*, 2024.
- Rein, D., Hou, B. L., Stickland, A. C., Petty, J., Pang, R. Y., Dirani, J., Michael, J., and Bowman, S. R. Gpqa: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024.
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y., Wu, Y., et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Singh, A., Co-Reyes, J. D., Agarwal, R., Anand, A., Patil, P., Garcia, X., Liu, P. J., Harrison, J., Lee, J., Xu, K., et al. Beyond human data: Scaling self-training for problem-solving with language models. *Transactions on Machine Learning Research*, 2023.
- Skean, O., Osorio, J. K. H., Brockmeier, A. J., and Giraldo, L. G. S. Dime: Maximizing mutual information by a difference of matrix-based entropies. *arXiv preprint arXiv:2301.08164*, 2023.

- Skean, O., Arefin, M. R., Zhao, D., Patel, N. N., Naghiyev, J., LeCun, Y., and Shwartz-Ziv, R. Layer by layer: Uncovering hidden representations in language models. In *Forty-second International Conference on Machine Learning*, 2025.
- Song, Y., Zhang, H., Eisenach, C., Kakade, S. M., Foster, D., and Ghai, U. Mind the gap: Examining the self-improvement capabilities of large language models. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Talmor, A., Herzig, J., Lourie, N., and Berant, J. Commonsenseqa: A question answering challenge targeting commonsense knowledge. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 2019.
- Team, K., Du, A., Gao, B., Xing, B., Jiang, C., Chen, C., Li, C., Xiao, C., Du, C., Liao, C., et al. Kimi k1. 5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*, 2025.
- Tong, Y., Zhang, X., Wang, R., Wu, R., and He, J. Dartmath: difficulty-aware rejection tuning for mathematical problem-solving. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, 2024.
- Wang, T., Li, S., and Lu, W. Self-training with direct preference optimization improves chain-of-thought reasoning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.
- Wang, W., Liu, X., Huang, B., Cui, H., Zhang, R., Yu, C., Jin, S., Yang, J., Yin, Q., Wang, Z., et al. Heapa: Difficulty-aware heap sampling and on-policy query augmentation for llm reinforcement learning. *arXiv preprint arXiv:2601.22448*, 2026.
- Wang, X., Wei, J., Schuurmans, D., Le, Q. V., Chi, E. H., Narang, S., Chowdhery, A., and Zhou, D. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023.
- Wei, J., Bosma, M., Zhao, V., Guu, K., Yu, A. W., Lester, B., Du, N., Dai, A. M., and Le, Q. V. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations*, 2022.
- Wen, L., Cai, Y., Xiao, F., He, X., An, Q., Duan, Z., Du, Y., Liu, J., Tanglifu, T., Lv, X., Zou, H., Deng, Y., Jia, S., and Zhang, X. Light-r1: Curriculum SFT, DPO and RL for long COT from scratch and beyond. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025.
- Wu, T., Li, X., and Liu, P. Progress or regress? self-improvement reversal in post-training. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Xiao, W. and Gan, L. Fast-slow thinking grpo for large vision-language model reasoning. *Advances in Neural Information Processing Systems*, 2026.
- Xu, F., Hao, Q., Zong, Z., Wang, J., Zhang, Y., Wang, J., Lan, X., Gong, J., Ouyang, T., Meng, F., et al. Towards large reasoning models: A survey of reinforced reasoning with large language models. *arXiv preprint arXiv:2501.09686*, 2025a.
- Xu, H., Wu, X., Wang, W., Li, Z., Zheng, D., Chen, B., Hu, Y., Kang, S., Ji, J., Zhang, Y., et al. Redstar: Does scaling long-cot data unlock better slow-reasoning systems? *arXiv preprint arXiv:2501.11284*, 2025b.
- Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H., et al. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025a.
- Yang, C., Si, Q., Duan, Y., Zhu, Z., Zhu, C., Li, Q., Lin, Z., Cao, L., and Wang, W. Dynamic early exit in reasoning models. *arXiv preprint arXiv:2504.15895*, 2025b.
- Yang, S., Tong, Y., Niu, X., Neubig, G., and Yue, X. Demystifying long chain-of-thought reasoning. In *Forty-second International Conference on Machine Learning*, 2025c.
- Yu, Q., Zhang, Z., Zhu, R., Yuan, Y., Zuo, X., Yue, Y., Dai, W., Fan, T., Liu, G., Liu, L., et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- Yuan, Z., Yuan, H., Li, C., Dong, G., Lu, K., Tan, C., Zhou, C., and Zhou, J. Scaling relationship on learning mathematical reasoning with large language models. *arXiv e-prints arXiv: 2308.01825*, 2023.
- Zelikman, E., Wu, Y., Mu, J., and Goodman, N. D. Star: self-taught reasoner bootstrapping reasoning with reasoning. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, 2022.
- Zhang, C., Yang, Y., Liu, J., Wang, J., Xian, Y., Wang, B., and Song, D. Lifting the curse of capacity gap in distilling language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023.

- Zhang, D., Wu, J., Lei, J., Che, T., Li, J., Xie, T., Huang, X., Zhang, S., Pavone, M., Li, Y., et al. Llama-berry: Pairwise optimization for olympiad-level mathematical reasoning via o1-like monte carlo tree search. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2025.
- Zhong, Q., Ding, L., Cai, X., Liu, J., Du, B., and Tao, D. Kaft: Knowledge-aware fine-tuning for boosting llms’ domain-specific question-answering performance. In *Findings of the Association for Computational Linguistics: ACL 2025*, 2025a.
- Zhong, Q., Ding, L., Liao, F., Liu, J., Du, B., and Tao, D. Resolving knowledge conflicts in domain-specific data selection: A case study on medical instruction-tuning. *arXiv preprint arXiv:2505.21958*, 2025b.
- Zhong, Q., Ding, L., Xuan, W., Liu, J., Du, B., and Tao, D. Learn to think: Improving multimodal reasoning through vision-aware self-improvement training. *arXiv preprint*, 2026a.
- Zhong, Q., Wang, K., Xu, Z., Ding, L., Liu, J., and Du, B. Achieving > 97% on gsm8k: Deeply understanding the problems makes llms better solvers for math word problems. *Frontiers of Computer Science*, 2026b.
- Zuo, Y., Qu, S., Li, Y., Chen, Z.-R., Zhu, X., Hua, E., Zhang, K., Ding, N., and Zhou, B. Medxpertqa: Benchmarking expert-level medical reasoning and understanding. In *Forty-second International Conference on Machine Learning*, 2025.

Appendix

Roadmap. In the part, we introduce the related work in Appendix A and the implementation of H-GRPO in Appendix B. Then, we provide the experimental details in Appendix C. Lastly, we present more experiments and analyses in Appendix D.

A. Related Work

Post-training for Reasoning. Recently, post-training the LLMs with explicit reasoning paths has shown remarkable potential to unleash their reasoning capabilities (Li et al., 2025; Laat et al., 2024; Zhong et al., 2026b). In the context of LRMs, common post-training paradigms include SFT (Wei et al., 2022; Xu et al., 2025b; Zhong et al., 2025a) and preference learning (Rafailov et al., 2023; Wen et al., 2025). Specifically, SFT encourages LLMs to mimic the reasoning behavior demonstrated in training samples (Xu et al., 2025b; Huang et al., 2024), while preference learning aims to align LLMs with human preferences for different reasoning trajectories (Zhang et al., 2025; Wen et al., 2025). Despite their remarkable performance, these methods are highly dependent on extensive, high-quality reasoning trajectories. Notably, although the new emerging RLVR training paradigm can also enhance models’ reasoning performance without relying on reasoning trajectories (Guo et al., 2025), cold-start training with these trajectories can improve training efficiency and yield higher performance (Yang et al., 2025c). This also underscores the importance of explicit reasoning trajectories. Besides obtaining these trajectories from human experts, a common alternative way is to distill the reasoning trajectories from a larger proprietary model, *e.g.*, DeepSeek-R1, which is still costly and time-consuming (Peng et al., 2025). More seriously, for some proprietary LLMs, their reasoning trajectories are even unavailable due to the licensing restrictions.

Self-improvement in Large Reasoning Models. To address the above issue, recent literature introduces the “self-improvement” paradigm, where models improve themselves using self-generated data without any external supervision (Zelikman et al., 2022; Yuan et al., 2023; Huang et al., 2023; Gulcehre et al., 2023; Wang et al., 2024; Hosseini et al., 2024; Wu et al., 2025; Huang et al., 2025a; Song et al., 2025). For instance, STaR (Zelikman et al., 2022) utilizes few-shot examples to gather self-synthesizing correct reasoning paths for SFT training, while RFT (Yuan et al., 2023) and ReST^{EM} (Singh et al., 2023) extend STaR by sampling multiple responses for each question. In the preference learning phase, IRPO (Pang et al., 2024) constructs preference pairs by using the self-generated correct responses as the pair winners and the incorrect responses as the pair losers. Similarly, Wang et al. (2024) show that self-training augmented by preference learning can improve the CoT reasoning capabilities of small-scale LLMs. However, we empirically find that these self-improvement methods usually suffer from data imbalance and overthinking problems (§2.2). Some prior studies also recognize these problems and attempt to address them by collecting more correct solutions for difficult queries (Tong et al., 2024; Ding et al., 2025; Koh et al., 2025; Wang et al., 2026; Chen et al., 2025b) and designing length-oriented reward functions to penalize too long solutions (Team et al., 2025; Yang et al., 2025c; Munkhbat et al., 2025; Dai et al., 2025a; Xiao & Gan, 2026). For instance, to alleviate the data imbalance, Tong et al. (2024) propose to allocate more trials to difficult queries, and Ding et al. (2025) leverage additional signals (*e.g.*, answers) to guide the reasoning of LRMs. AdaSTaR (Koh et al., 2025) proposes an adaptive sampling strategy to ensure data balance by prioritizing under-trained examples. While effective, these methods overlook the reuse of prior failed solutions and require a larger inference budget. On the other hand, to mitigate the overthinking, S-GRPO (Dai et al., 2025b) performs Early-exit Thought Rollout at different reasoning steps to reduce redundant reasoning. Despite its effectiveness, it introduces substantial computational overhead during training, which is inefficient. FAST-GRPO (Xiao & Gan, 2026) and GRPO- λ (Dai et al., 2025a) design length-oriented rewards to penalize redundant reasoning, similar to the Long2Short (Team et al., 2025) and CosFn (Yang et al., 2025c) baselines in our experiments, which may suffer from over-penalizing length and consequent performance degradation (Dai et al., 2025b).

Different from prior studies, we propose two simple-yet-effective approaches to address these problems efficiently. Specifically, instead of solely allocating more trials to difficult queries, our proposed *VeriExit* strategy proposes to reuse partial correct reasoning steps from previous failed solutions to improve the sampling efficiency. It is noteworthy that the technology of *VeriExit* bears some resemblance to prior early-exit decoding methods (Rahmath P et al., 2024; Yang et al., 2025b), but the idea of reusing previous failed solutions for efficient data synthesis is innovative. On the other hand, to alleviate overthinking, we introduce the *InDiv* score that leverages the internal state of LRMs as a signal to encourage models’ diverse and concise reasoning, rather than simply using a length penalty. To the best of our knowledge, our *InDiv* is one of the first works that use the internal states of LRMs to guide the concise reasoning during self-improvement training.

Notably, our work differs from the concurrent work (Zhong et al., 2026a) in three key aspects: 1) *Different research objectives.* The concurrent work focuses on self-improvement in multimodal LLMs and mainly addresses the visual hallucinations issue, whereas we study self-improvement in LRMs and target the overthinking problem specific to long-CoT

reasoning. 2) *Different methodologies*. Although both works involve alleviating data imbalance, the concurrent work requires two model feed-forward passes, leading to more computation overhead, while our method leverages a more efficient text-matching method to identify the partially correct reasoning steps. 3) *Different experimental settings*. The concurrent work evaluates on visual-oriented tasks, whereas we conduct experiments on several text-only reasoning tasks.

B. Implementation of H-GRPO

Background of GRPO and RLVR. Group Relative Policy Optimization (GRPO) (Shao et al., 2024) is a popular RL algorithm, which is widely used in the current popular RLVR training paradigm. Formally, let $\mathcal{M}_{\theta_{\text{ref}}}$ and $\mathcal{M}_{\theta_{\text{new}}}$ denote the reference model and current policy model, GRPO samples a group of solutions $\{(\tilde{r}_i^g, \tilde{y}_i^g)\}_{g=1}^G$ for each query $x_i \in \mathcal{D}$, where G denotes the number of solutions in a group. For ease of description, we simplify the solution $(\tilde{r}_i^g, \tilde{y}_i^g)$ as a_i^g . Then, we can optimize the $\mathcal{M}_{\theta_{\text{new}}}$ by maximizing the GRPO objective. Notably, inspired by Yu et al. (2025) who use a token-level policy gradient loss to address the unhealthy increase in response length problem of the vanilla GRPO method, we employ an improved token-level GRPO objective function as:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}[x_i \in \mathcal{D}, \{a_i^g\}_{g=1}^G \sim \mathcal{M}_{\theta_{\text{ref}}}(\cdot|x_i)] \frac{1}{\sum_{g=1}^G |a_i^g|} \sum_{g=1}^G \sum_{o=1}^{|a_i^g|} \left(\min(z_o(a_i^g|x_i) A_g, \text{clip}(z_o(a_i^g|x_i), 1 - \epsilon, 1 + \epsilon) A_g) - \gamma \mathbb{D}_{\text{KL}}(\mathcal{M}_{\theta_{\text{new}}} || \mathcal{M}_{\theta_{\text{ref}}}) \right), \quad (6)$$

$$z_o(a_i^g|x_i) = \frac{\mathcal{M}_{\theta_{\text{new}}}(a_{i,o}^g|x_i, a_{i,<o}^g)}{\mathcal{M}_{\theta_{\text{ref}}}(a_{i,o}^g|x_i, a_{i,<o}^g)}, \quad (7)$$

where ϵ and γ are hyper-parameters set to 0.2 and 0.04, respectively, $\mathbb{D}_{\text{KL}}$ is a KL penalty term, and A_g is the advantage computed as follows:

$$A_g = \frac{R_g - \text{mean}(\{R_1, R_2, \dots, R_G\})}{\text{std}(\{R_1, R_2, \dots, R_G\})}, \quad (8)$$

where R_g denotes the outcome reward of g -th ($g \in [1, G]$) solution in the group. In the reasoning tasks that contain clear and verifiable answers, e.g., mathematical reasoning, the reward R_g mainly consists of two types of rewards:

- **Accuracy reward** R^{accuracy} : It evaluates whether the solution is correct, i.e., $\mathbb{I}(\tilde{y}_i^g, y_i) = 1$.
- **Format reward** R^{format} : It evaluates whether the defined tags are present in the final solution, i.e., '<think>' and '</think>', '<answer>' and '</answer>' tags.

Reward of H-GRPO. To alleviate the overthinking problem in the vanilla GRPO, we propose H-GRPO, which improves the GRPO by leveraging our *InDiv* scores as an extra reward. Specifically, for each solution in a group, we calculate its *InDiv* score as Eq. 5, and further normalize the score as:

$$R_g^{\text{InDiv}} = \frac{\text{InDiv}^g}{\max(\{\text{InDiv}^1, \dots, \text{InDiv}^G\})}, \quad (9)$$

where $\max(\cdot)$ denotes the maximum *InDiv* scores in a group. The final reward for H-GRPO is the combination of all rewards:

$$R_g^{\text{all}} = R_g^{\text{accuracy}} + R_g^{\text{format}} + \omega \cdot R_g^{\text{InDiv}}, \quad (10)$$

where ω is a coefficient to control the weight of R_g^{InDiv} , which is set to 0.2 in our experiments. Notably, since the accuracy reward weight (1.0) far exceeds the *InDiv* reward weight (0.2), any attempt by the model to inflate the *InDiv* score by inserting irrelevant content would likely produce incorrect answers, resulting in a net negative reward (approximately -0.8), which inherently suppresses the incentive for reward hacking.

More analyses of Table 5. We present the results of Qwen2.5 models trained with different GRPO methods in Table 5. Some readers may wonder why the output length of the tuned model significantly decreases after GRPO training, compared to the initial SFT, i.e., from 1,540 to 987 average tokens. We conjecture that there are two main reasons. On the one hand, the token-level loss function used in the improved GRPO algorithm (Eq. 6) can effectively alleviate the abnormal increase in response length (Yu et al., 2025). On the other hand, according to the public experimental record² of ms-swift³, during the

²<https://swift.readthedocs.io/en/latest/BestPractices/GRPO.html#grpo-training-experiment-record>

³<https://github.com/modelscope/ms-swift>

Algorithm 1 Reasoning Diversity Metric

```

1: Input: self-generated reasoning trajectory  $\hat{r}_i^k = [\hat{r}_{i,1}^k, \dots, \hat{r}_{i,L}^k]$ , similarity threshold  $\tau_{\text{sim}}$ 
2: Output: reasoning diversity score  $\rho_i^k$ 
3: Initialize Empty Unfiltered Set  $\mathcal{U}$ 
4: for Each reasoning step  $\hat{r}_{i,l}^k \in \hat{r}_i^k$  do
5:   Obtaining the sentence embedding  $\text{emb}(\hat{r}_{i,l}^k)$  using the BGE-m3 model
6:   //  $\text{Cos}(\text{emb}(\hat{r}_{i,l}^k), \mathcal{U})$  denotes the cosine distance between  $\text{emb}(\hat{r}_{i,l}^k)$  and its nearest neighbor in  $\mathcal{U}$ 
7:   if  $\text{Cos}(\text{emb}(\hat{r}_{i,l}^k), \mathcal{U}) < \tau_{\text{sim}}$  then
8:      $\mathcal{U} \leftarrow \mathcal{U} \cup \hat{r}_{i,l}^k$ 
9:   else
10:    Continue
11:   end if
12: end for
13: Return:  $\rho_i^k = \frac{|\mathcal{U}|}{|\{\hat{r}_{i,1}^k, \dots, \hat{r}_{i,L}^k\}|}$ 
    
```

GRPO training, the solution length initially decreases and then increases, indicating that the model changed its reasoning manner. Since the GRPO training is computationally expensive, we do not train the models for very long steps. That is, the training of our models may still be in the stage of decreasing output length. Despite all this, our H-GRPO can further reduce the inference overhead, while achieving better reasoning performance. These results can prove the superiority of H-GRPO, and we believe that it has great potential to perform better after longer GRPO training.

Combining H-GRPO with HSIR. Here, we investigate whether H-GRPO can be combined with HSIR. Intuitively, HSIR-SFT training yields a stronger cold-start model, which can then serve as the policy model for subsequent H-GRPO training. To validate this hypothesis, we initialize H-GRPO with the HSIR-SFT (Iteration 1) model. Results for the Qwen2.5 family on GSM8K are presented in Table 7. The combination consistently outperforms either method alone across all model scales, confirming that the two approaches are complementary. Nevertheless, it is worth noting that combining H-GRPO with HSIR incurs additional computational overhead. Exploring more efficient strategies for integrating these two methods is a promising direction for future work.

Table 7. Results of combining HSIR-SFT with H-GRPO on GSM8K benchmark across Qwen2.5 family models.

Method	Qwen2.5-1.5B	Qwen2.5-3B	Qwen2.5-7B
HSIR-SFT	71.72	86.13	88.78
H-GRPO	74.60	83.09	91.43
HSIR-SFT + H-GRPO	76.72	87.11	91.58

C. More Experimental Details

C.1. Dataset Details

In this work, we evaluate the trained models on several representative and challenging reasoning benchmarks. Here, we introduce the descriptions of these tasks as follows:

- **MedQA:** MedQA (Jin et al., 2021) is a challenging medical question-answering task, which consists of questions and corresponding 4-option or 5-option answers in the style of the US Medical License Exam (USMLE). Since the original MedQA does not contain the reasoning trajectories, we prompt the DeepSeek-R1 to generate the reasoning data. The prompt is shown in Table 8, and Table 9 presents a case of distilled reasoning trajectories. For evaluation, we follow prior works (Chen et al., 2023; Zhong et al., 2025b) and use the 4-option MedQA with 1,273 samples as the test set.
- **GSM8K:** GSM8K (Cobbe et al., 2021) is a widely-used mathematical reasoning task, which contains 8.5K high-quality grade school math word problems. Since the original GSM8K does not contain any reasoning trajectories, we alternatively use the GSM8K version⁴ released by CAMEL (Li et al., 2023). Notably, the dataset is also distilled from the DeepSeek-R1. Table 10 presents a case of distilled GSM8K training data. For in-distribution evaluation, we directly use the original GSM8K with 1.32K test samples.
- **Medbullets:** Medbullets (Chen et al., 2025a) comprises 308 difficult USMLE Step 2&3 style medical questions collected

⁴https://huggingface.co/datasets/camel-ai/gsm8k_distilled

Algorithm 2 Self-improvement Training with HSIR

```

1: Input: base model  $\mathcal{M}_{base}$ , seed data  $\mathcal{S} = \{(x_i, r_i, y_i)\}_{i=1}^N$ , unlabeled dataset  $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^M$ 
2: Output: self-improved model  $\mathcal{M}_T$ 
3: Fine-tune  $\mathcal{M}_{base}$  on  $\mathcal{S}$  to get initial reasoning model  $\mathcal{M}_0$ 
4: for  $t \in [1, T]$  do
5:   # Self-generation
6:   Obtain  $K$  solutions  $\{(\hat{r}_i^k, \hat{y}_i^k)\}_{k=1}^K$  generated by  $\mathcal{M}_{t-1}$  for each  $x_i \in \mathcal{D}$ 
7:   Verify the correctness of self-generated solutions, and split them into two groups:
      $\hat{\mathcal{D}}_t^{correct} = \{(x_i, \hat{r}_i^k, \hat{y}_i^k) \mid x_i \in \mathcal{D}; k \in [1, K]; \mathbb{I}(\hat{y}_i^k, y_i) = 1\}$ 
      $\hat{\mathcal{D}}_t^{wrong} = \{(x_i, \hat{r}_i^k, \hat{y}_i^k) \mid x_i \in \mathcal{D}; k \in [1, K]; \mathbb{I}(\hat{y}_i^k, y_i) = 0\}$ 
8:
9:   # VeriExit sampling process
10:  for Each sample  $(x_i, \hat{r}_i^k, \hat{y}_i^k) \in \hat{\mathcal{D}}_t^{wrong}$  do
11:    for Each reasoning step  $\hat{r}_{i,l}^k \in [\hat{r}_{i,1}^k, \dots, \hat{r}_{i,L}^k]$  do
12:      if  $\hat{r}_{i,l}^k$  arrives at  $y_i$  then
13:        Obtain a new query " $x_i + [\hat{r}_{i,1}^k, \dots, \hat{r}_{i,l}^k] + \backslash n \backslash n < / think > \backslash n < answer > \backslash n$ "
14:        Feed the new query into  $\mathcal{M}_{t-1}$  to resample  $J$  answers  $\{\hat{y}_i^{k,j}\}_{j=1}^J$ 
15:        Break
16:      else
17:        Continue
18:      end if
19:    end for
20:    Merge the correct solutions ( $\mathbb{I}(\hat{y}_i^{k,j}, y_i) = 1$ ) into  $\hat{\mathcal{D}}_t^{correct}$ 
21:  end for
22:
23:  # Calculate the InDiv score
24:  for Each query  $x_i \in \hat{\mathcal{D}}_t^{correct}$  do
25:    Calculate the InDiv score  $\mathbf{InDiv}_i^k$  for each correct solution  $(\hat{r}_i^k, \hat{y}_i^k)_{k=1}^K$  as Eq. 5
26:    Update the dataset  $\hat{\mathcal{D}}_t^{correct}$  by filtering the undesired solution with  $\mathbf{InDiv}_i^k < \tau$ 
27:  end for
28:
29:  # SFT Training
30:  Fine-tune  $\mathcal{M}_{base}$  with  $\mathcal{L}_{SFT}$  in Eq. 1 on the combination of  $\mathcal{S}$  and  $\hat{\mathcal{D}}_t^{correct}$ 
31:  # or DPO Training
32:  Obtain a pairwise dataset  $\hat{\mathcal{D}}_t^{pairs} = \{(x_i, \hat{r}_i^{k_w}, \hat{y}_i^{k_w}), (x_i, \hat{r}_i^{k_l}, \hat{y}_i^{k_l}) \mid x_i \in \hat{\mathcal{D}}_t; k_w, k_l \in [1, K]\}$ ,
    where  $(\hat{r}_i^{k_w}, \hat{y}_i^{k_w}) \sim \hat{\mathcal{D}}_t^{correct}$  and  $(\hat{r}_i^{k_l}, \hat{y}_i^{k_l}) \sim \hat{\mathcal{D}}_t^{wrong}$ 
33:  Continually train  $\mathcal{M}_{t-1}$  with  $\mathcal{L}_{DPO+NLL}$  in Eq. 2 on  $\hat{\mathcal{D}}_t^{pairs}$ 
34: end for

```

from real-world conversations. Each question is paired with a case description and multiple answer choices. In our work, we use the 4-option Medbullets as the OOD test set of LRMs trained on MedQA.

- **MATH:** MATH (Hendrycks et al., 2021) comprises 500 problems spanning five core mathematical domains: algebra, combinatorics, geometry, number theory, and precalculus. Each problem is designed to test the multi-step and complex reasoning abilities of LRMs, requiring more than simple calculation or knowledge recall. In our experiments, we use this challenging dataset to evaluate the OOD performance of models trained on GSM8K.
- **MedXpertQA:** MedXpertQA (Zuo et al., 2025) contains 4,460 high-difficulty medical questions spanning 17 specialties and 11 body systems. It includes two subsets, MedXpertQA Text for text medical evaluation and MedXpertQA MM for multimodal medical evaluation. We use the MedXpertQA Text as the OOD test set of LRMs trained on MedQA.
- **AMC2023:** AMC2023 (Mathematical Association of America, 2023) consists of 40 challenging mathematical problems from American Mathematics Competitions, which is widely used to evaluate the complex reasoning performance of LRMs. We use the public test set⁵ to evaluate the OOD performance of models trained on GSM8K.
- **ARC:** AI2 Reasoning Challenge’s challenge set (ARC) (Clark et al., 2018) is a scientific reasoning dataset that contains 1.12K multiple-choice science QA training samples and 1.17K test samples. Similar to MedQA, we prompt the DeepSeek-

⁵<https://huggingface.co/datasets/zwhe99/amc23>

R1 to generate the reasoning steps for the training samples, and randomly select 500 samples as the seed data, using the remaining training samples as unlabeled data. The trained models are evaluated on the ARC test set.

- **CommonsenseQA:** CommonsenseQA (CSQA) (Talmor et al., 2019) is a multiple-choice question-answering dataset that requires diverse types of commonsense knowledge to predict the correct answers. It contains 12,102 questions, each with one correct answer and four distractors. Similarly, for the 9.74K training samples, we distill reasoning steps from DeepSeek-R1 and randomly select 1K samples as seed data. The trained models are evaluated on the 1.14K test samples.
- **StrategyQA:** StrategyQA (StraQA) (Geva et al., 2021) is an implicit multi-hop reasoning benchmark, containing 1.6K training samples and 687 test samples. After distilling the reasoning steps from DeepSeek-R1 for the training samples, we randomly select 500 training samples as the seed data. The trained models are directly evaluated on the test samples.

C.2. Training and Evaluation Details

In the SFT phase, we fine-tune each model with a batch size of 32 and a peak learning rate of $1e-5$, except $2e-6$ for 7B/8B models. In the DPO phase, the batch size is set to 16, and the peak learning rates for smaller (1.5B/3B) models and larger (7B/8B) models are set to $1e-6$ and $2e-7$, respectively. In both training settings, the warm-up ratio is 0.1, and the maximum tokenizer length is 2,048. All models are trained with full parameter fine-tuning for 3 epochs. We use the popular LLaMA-Factory⁶ toolkit as the training codebase. For the RLVR training, we implement the GRPO with the open-r1⁷ framework. The roll-out size for each query is set to 8, the learning rate is set to $1e-6$, and the batch size is set to 14. Since the GRPO training is time-consuming and costly, we only train the models for 300 steps. For implementing our H-GRPO, the ω that controls the weight of R^{InDiv} is set to 0.2. All experiments are conducted on 8 NVIDIA A800 (80GB) GPUs.

During evaluation, we sample the model responses by using the greedy decoding method for reproducibility. The maximum output length is set 4,096. All models are evaluated in a zero-shot manner. We extract the final answer from the model response according to the given format. If no valid answer is extracted, the response is considered incorrect.

Table 8. Prompts used for distilling reasoning paths for MedQA from DeepSeek-R1.

System Prompt:

You are a helpful medical AI Assistant. Your task is to select one correct answer from A to D for the medical question. Please respond in the following format: ‘##Explanation:\n[explanation]\n\n##Answer:\n[option]’

User Prompt:

For the following medical question, select one correct answer from A to D.

Question: {Question}

Options:

A. {Option-A}

B. {Option-B}

C. {Option-C}

D. {Option-D}

Table 9. An example of seed data in MedQA distilled from DeepSeek-R1.

System Prompt:

You are a helpful medical AI Assistant that provides well-reasoned and detailed responses for medical questions. You first think about the reasoning process as an internal monologue and then provide the user with the concise and accurate answer. Respond in the following format: <think>\n...\n</think>\n<answer>\n...\n</answer>

Instruction:

For the following medical question, select one correct answer from A to D.

⁶<https://github.com/hiyouga/LLaMA-Factory>

⁷<https://github.com/huggingface/open-r1>

Question: A 55-year-old woman comes to the physician because of fevers for 2 weeks. She works as a nurse and recently returned from a charity work trip to India, where she worked in a medically-underserved rural community. A tuberculin skin test 3 months ago prior to her trip showed an induration of 3 mm. Physical examination is unremarkable. An x-ray of the chest shows right-sided hilar lymphadenopathy. A sputum culture shows acid-fast bacilli. Which of the following immunologic processes most likely occurred first?

Options:

- A. Formation of a nodular tubercle in the lung
- B. Replication of bacteria within alveolar macrophages
- C. Production of interferon-gamma by T-helper cells
- D. Migration of T-helper cells to the lungs

Please respond in the following format: '##Explanation:\n[explanation]\n\n##Answer:\n[option]'

Output:

<think> *#reasoning trajectory*

Okay, let's tackle this question step by step. The patient is a 55-year-old nurse who recently returned from India and has symptoms of TB. Let's break down the information given.

She had a tuberculin skin test (TST) three months ago with 3 mm induration, which is considered negative (since the cutoff for healthcare workers is usually 10 mm). Now, she has a positive sputum culture for acid-fast bacilli, which points to active tuberculosis. The chest X-ray shows hilar lymphadenopathy, which is common in primary TB, especially in the right side.

The question is asking about the first immunologic process that occurred. The options are about the sequence of immune responses when someone is infected with *Mycobacterium tuberculosis*.

So, the initial step when TB bacteria enter the body is inhalation. The bacilli reach the alveoli, where they are taken up by alveolar macrophages. However, *M. tuberculosis* has mechanisms to survive inside these macrophages. The bacteria can replicate within the macrophages until the macrophages are overwhelmed and lyse, releasing more bacteria. This replication within macrophages is an early step before the immune system starts to respond more effectively.

After that, the adaptive immune response kicks in. T-helper cells (CD4+ T cells) migrate to the site of infection. These cells release cytokines like interferon-gamma, which activate macrophages to become more effective at killing the bacteria. The activated macrophages form granulomas (tubercles) to contain the infection. The formation of the granuloma is part of the body's attempt to wall off the infection.

So the sequence would be: bacteria enter macrophages and replicate (B), then T-cells migrate (D), produce interferon-gamma (C), leading to granuloma formation (A). The question asks which happened first. So the first step after exposure would be the replication within macrophages. The TST was negative before her trip, suggesting she wasn't previously infected. Now she's positive, so this is a new infection. The initial infection's first immunologic process is the bacteria replicating in macrophages before the immune response starts.

</think>

<answer> *#answer*

##Explanation:

The patient's recent exposure and subsequent active tuberculosis indicate a new infection. The initial immunologic process begins when *Mycobacterium tuberculosis* is inhaled and phagocytosed by alveolar macrophages. The bacteria survive and replicate within these macrophages (option B), which occurs before the adaptive immune response. Subsequent steps include T-helper cell migration to the lungs (D), interferon-gamma production (C), and granuloma formation (A). The replication within macrophages is the earliest event in the immunologic cascade.

##Answer:

B

</answer>

Table 10. An example of seed data in GSM8K distilled from DeepSeek-R1.

System Prompt:

You are a helpful AI Assistant that provides well-reasoned and detailed responses for the math word problems. You first think about the reasoning process as an internal monologue and then provide the user with the concise and accurate answer. The final answer should be provided in the `\boxed{}`.

Instruction:

John starts at an elevation of 400 feet. He travels downward at a rate of 10 feet down per minute for 5 minutes. What is his elevation now?

Output:

<think> *#reasoning trajectory*

Okay, so I need to figure out John's elevation after he travels downward for 5 minutes at a rate of 10 feet per minute. Let me start by understanding the problem. John starts at 400 feet, and he's moving downward. Moving downward would decrease his elevation, right? So, he's going from higher elevation to lower.

First, I should note his starting point: 400 feet. Then, he's going down at 10 feet per minute for 5 minutes. So, I think I can calculate the total distance he descended by multiplying the rate by the time. That makes sense because if you go 10 feet every minute, then in 5 minutes, that's $10 * 5 = 50$ feet. Wait, yes, so he would have gone down 50 feet in total.

Since he's moving downward, his new elevation will be the original elevation minus that descent. So, subtracting 50 feet from 400 feet. Let me do that calculation: $400 - 50$. That gives 350 feet. Hmm, so his elevation after 5 minutes should be 350 feet.

Wait, let me check that again. Multiplication first: 10 feet per minute times 5 minutes is indeed 50 feet. Then subtract that from the starting elevation. Yeah, 400 minus 50 is 350. I think that's correct. Maybe I should visualize this. Imagine he's on a hill at 400 feet. Each minute, he goes down 10 feet, so after 1 minute, he's at 390, then 380, 370, 360, 350 after 5 minutes. Each step is 10 feet lower. So yes, five steps would be 50 feet total.

Is there any chance I might have mixed up the direction? Like thinking he's ascending? But the problem clearly states he's traveling downward. So subtracting is the right operation. Okay, I feel confident that this is right. Let me just go through the steps again to confirm. Starting elevation 400. Rate: 10 ft/min downward. Time: 5 min. Total descent is $rate * time = 10 * 5 = 50$ ft. New elevation: $400 - 50 = 350$ ft. Yeah, that checks out. I think that's the correct answer.

</think>

#answer

John starts at an elevation of **400 feet**.

To find his elevation after descending:

1. **Calculate total descent**: $Rate = 10 \text{ feet/minute} \times 5 \text{ minutes} = 50 \text{ feet}$.
2. **Subtract the descent from the starting elevation**: $400 \text{ feet} - 50 \text{ feet} = 350 \text{ feet}$.

Verification: Each minute, his elevation decreases by 10 feet:

- After 1 minute: $400 - 10 = 390$ feet
- After 2 minutes: $390 - 10 = 380$ feet
- Continuing this pattern for 5 minutes leads to **350 feet**.

Final Answer:

`\boxed{350}`

D. More Experiments and Analyses

D.1. Analysis of *VeriExit* Sampling Strategy

Reliability of *VeriExit*. In *HSIR*, we propose *VeriExit* to efficiently collect more successful solutions for difficult queries by reusing partial correct reasoning steps of previous failed solutions. Here, to verify the reliability of this idea, we calculate the coverage of these verifiable successful solutions, which contain intermediate reasoning steps that arrived at the correct answer. Figure 5 (Left) illustrates the coverage of these solutions in the most difficult queries of MedQA. Specifically, we refer to the queries that did not obtain any correct solutions during the K -times ($K = 10$ in our experiments) self-generation processes as the most difficult ones. The x-axis denotes the number of verifiable successful solutions in a query, and the y-axis denotes the proportion of queries. As seen, among all Qwen2.5 family models, nearly 70% of these most difficult queries have at least one verifiable successful solution, and more than 10% of these queries have 4 or more verifiable successful solutions. We believe that in the simpler queries, there is a larger proportion of previous failed solutions that can be converted into verifiable successful solutions by *VeriExit*. These results can demonstrate the reliability of our *VeriExit*.

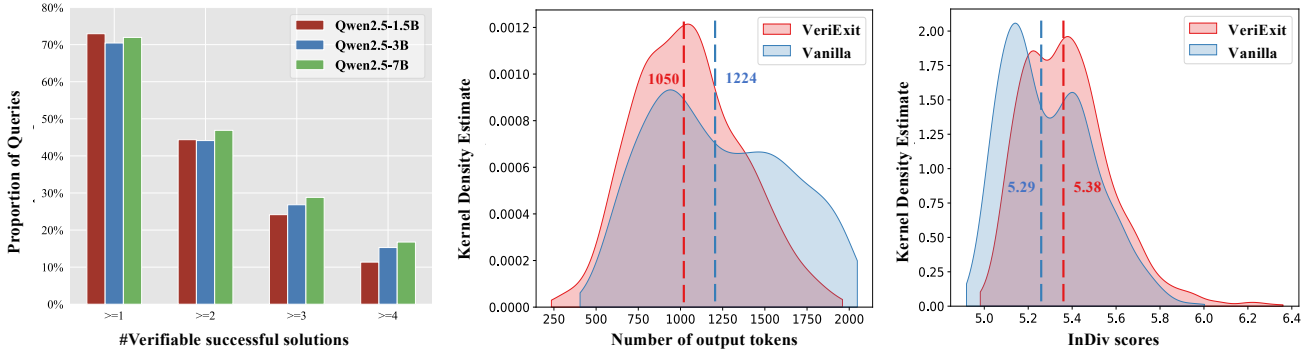


Figure 5. **Left:** Coverage of verifiable successful solutions. The x-axis denotes the number of verifiable successful solutions in a query. **Middle:** Distribution of the number of output tokens of correct self-generated solutions. **Right:** Distribution of *InDiv* scores of correct self-generated solutions. In the middle and right sub-figures, we use the initial SFT Qwen2.5-1.5B models.

Moreover, we compare the solutions resampled by *VeriExit* and the previous correct solutions. In practice, we measure the length and unnormalized *InDiv* scores of solutions on the difficult MedQA queries (obtaining four correct solutions during the previous self-generation) generated by initial SFT Qwen2.5-1.5B models, and visualize the distributions in Figure 5 (Middle and Right). As seen, compared to the previous correct solutions generated by the vanilla sampling strategy, our *VeriExit* can sample shorter and more concise reasoning trajectories. We attribute it to the self-truncation and early-exit processes of *VeriExit*, which can skip the redundant and repetitive reasoning steps. This can also explain why the “-w/ *VeriExit*” method in Table 3 can improve reasoning efficiency against the ReST^{EM} .

More *VeriExit* variants. The core of our *VeriExit* is to determine whether the reasoning step has reached the ground-truth answer. There are several methods to achieve this goal. Specifically, the simplest approach is to directly match the text between the reasoning output and the ground-truth answer. If the reasoning output explicitly mentions the answer, *e.g.*, “answer is $\{y_i\}$ ”, we can assume that it arrives at the ground-truth answer. Beyond this simple heuristic, inspired by SelfCheckGPT (Manakul et al., 2023), we can further use two more sophisticated approaches: **NLI-based** and **prompt-based**. In the **NLI-based *VeriExit***, we use an external Natural Language Inference (NLI) model to judge the relationships between the reasoning output and answer. The NLI model can determine whether the reasoning output entails the answer according to the similarity of sentence representations, *i.e.*, $\mathbb{I}(\text{NLI}(\hat{r}_{i,l}^k, y_i) = \text{entailment})$. In practice, we use the powerful DeBERTa-v3-large-mnli⁸ as the NLI model. In the **prompt-based *VeriExit***, we query the current $\mathcal{M}_{t-1}$ to assess whether the reasoning output and answer convey the same meaning by using the following prompt: “You are a semantic-equivalence classifier. Your only goal is to decide whether the two input sentences convey the same meaning. Please direct output the answer following the format: ‘##Answer: [YES|NO|UNCLEAR]’”. Notably, for both methods, we convert the answer into a full sentence using the template “the answer is y_i ” to ensure consistent semantic comparison.

To evaluate different *VeriExit* strategies, we measure the coverage of verifiable successful solutions on the most difficult queries of MedQA using each strategy. Using the text-matching method as the baseline, we illustrate the correlation between

⁸<https://huggingface.co/MoritzLaurer/DeBERTa-v3-large-mnli-fever-anli-ling-wanli>

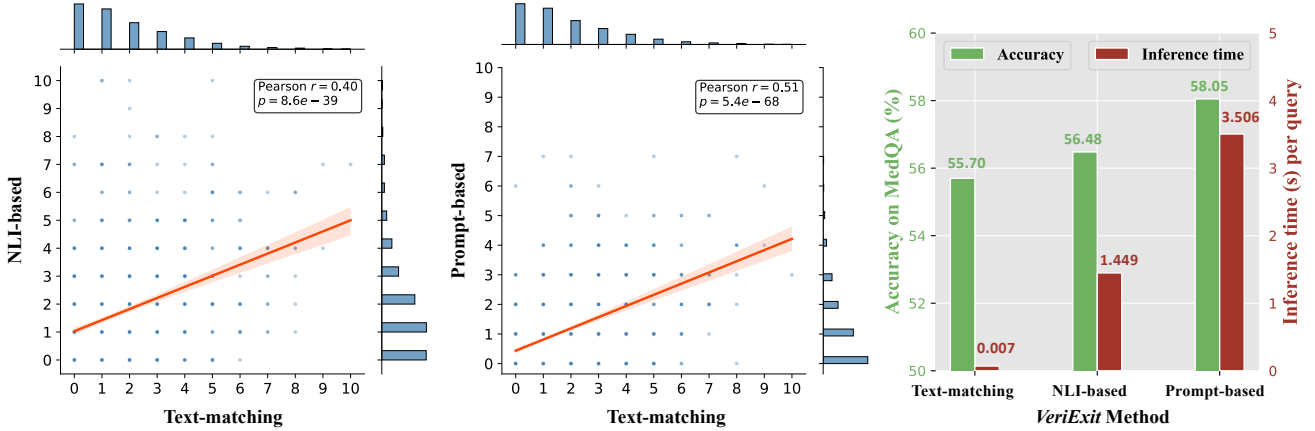


Figure 6. **Left:** Correlation between text-matching and NLI-based *VeriExit* methods. **Middle:** Correlation between text-matching and prompt-based *VeriExit* methods. **Right:** Performance and efficiency comparisons of HSIR-SFT variants equipped with different *VeriExit* methods. In the left and middle sub-figures, the axes denote the number of verifiable successful solutions per query. Qwen2.5-3B model is used in this experiment, and all models are self-improved for one iteration.

text-matching and NLI-based/prompt-based methods in Figure 6 (Left) and (Middle), respectively. In this experiment, we use the Qwen2.5-3B as the base model. The results show that text-matching *VeriExit* correlates well with both alternative methods, with Pearson Correlation Coefficients exceeding 0.4 and p-values below 0.05, indicating that all *VeriExit* variants produce largely consistent predictions. Further, we replace the *VeriExit* strategy in the HSIR-SFT framework, and compare the performance and efficiency of HSIR-SFT variants in Figure 6 (Right). The findings are as follows: 1) both NLI-based and prompt-based methods achieve better reasoning performance, as they can more accurately identify the correct reasoning steps during *VeriExit*; 2) although effective, these methods incur additional inference overhead. Therefore, for simplicity and efficiency, we use the text-matching method in our work by default.

D.2. Analysis of *InDiv* Score

Correlation between *InDiv* scores and semantic entropy. Here, we investigate the correlation between our *InDiv* scores and the semantic entropy of hidden representations $\mathbf{H}$. First, we introduce the **Matrix-Based Entropy** (Giraldo et al., 2014; Skean et al., 2023; 2025), which is a famous information-theoretic quantity. For the cross-covariance matrix $\Sigma_i^k = \mathbf{H}_i^k \cdot \mathbf{J}_d \cdot \mathbf{H}_i^k$ and its eigenvalues $\{\lambda_{i,u}^k\}_{u=1}^m$, the matrix-based entropy of order $\alpha > 0$ is:

$$S_{\alpha,i}^k(\mathbf{H}_i^k) = \frac{1}{1-\alpha} \log \left[\sum_{u=1}^m \left(\frac{\lambda_{i,u}^k}{\text{Tr}(\mathbf{H}_i^k)} \right)^\alpha \right], \quad (11)$$

where $\text{Tr}(\cdot)$ denotes the trace operator obtained from the sum of α -power of each eigenvalues (Horn & Johnson, 2012). When $\alpha \rightarrow 1$, the entropy $S_{\alpha,i}^k(\mathbf{H}_i^k)$ corresponds to the Shannon’s entropy of hidden representations. Intuitively, if the eigenvalues of $\mathbf{H}$ are in a uniform distribution, the entropy will be higher, indicating that $\mathbf{H}$ contains more diverse features (Skean et al., 2025). Conversely, if the eigenvalues collapse to a small area, the entropy will be smaller. To verify it, we compare the distributions of eigenvalues between the concise and overthinking solutions identified by our *InDiv* scores. Specifically, we directly use the solutions in Table 18, and illustrate their eigenvalue distributions in Figure 7 (Left). It can be found that the eigenvalue distributions of the concise solution are more uniform than those of the overthinking solution. To have a closer look, we directly visualize the hidden representations of both solutions. Figure 7 (Middle) and (Right) show the t-SNE results of the concise and overthinking solutions, respectively. We can observe that the distribution of hidden representations of the concise solution is more diverse and uniform, while that of the overthinking solution is more similar and concentrated. Overall, these results indicate that a higher *InDiv* score usually refers to a higher semantic entropy of hidden representations, which effectively proves why our *InDiv* score can help identify overthinking solutions.

Impact of layer depth for calculating *InDiv* scores. As mentioned in §3, we use the hidden representations from the middle layer of $\mathcal{M}_{t-1}$ to calculate the *InDiv* scores. Here, we investigate the impact of different layer depths by comparing the performance of Qwen2.5-1.5B trained with different HSIR-SFT configurations on MedQA. Specifically, since the

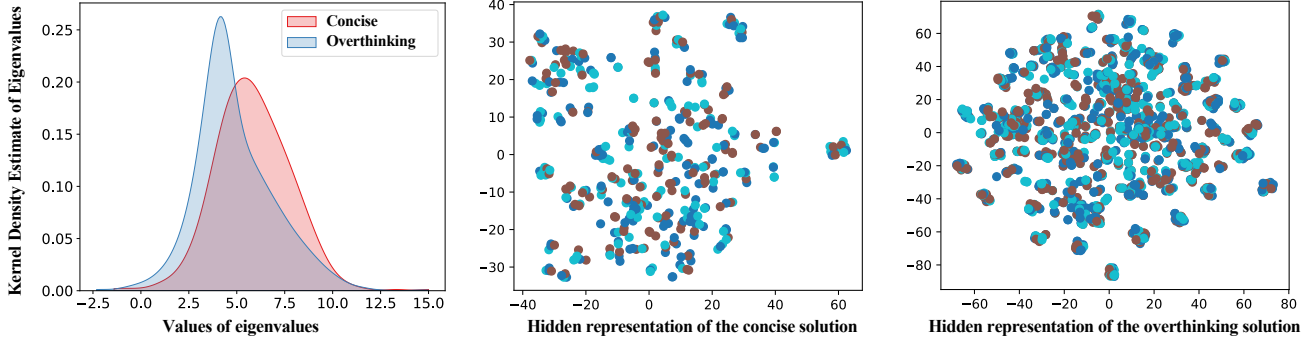


Figure 7. **Left:** Distribution of eigenvalues of hidden representation in the concise and overthinking solutions. **Middle:** t-SNE visualizations of hidden representations in the concise solution. **Right:** t-SNE visualizations of hidden representations in the overthinking solution. Here, we use the initial SFT Qwen2.5-1.5B as the test model. The concise and overthinking solutions are from Table 18.

Table 11. **Analysis of important components in *InDiv*.** All models are self-improved for one iteration.

Method	Qwen2.5-1.5B		Qwen2.5-3B	
	MedQA	GSM8K	MedQA	GSM8K
SFT-Initial	38.10	63.99	49.02	77.18
HSIR-SFT	45.33	71.72	55.70	86.13
-w/o attention-aware	45.20	71.34	55.38	85.75
-w/o normalization	43.52	71.27	54.13	85.60

Table 12. **Comparison of data selection methods.** Results on Qwen2.5-1.5B after one iteration.

Method	MedQA	GSM8K
<i>InDiv</i> (Ours)	45.33	71.72
<i>Data selection methods</i>		
PPL-based	41.56	70.57
Quality-based	43.25	70.66
Entropy-based	41.24	70.66

Qwen2.5-1.5B contains 28 layers, we vary the layer used for calculating *InDiv* scores across $\{5, 10, 15, 20, 25\}$ and illustrate the comparative results in Figure 8 (Left). For reference, we also include the results of SFT-Initial and ReST^{EM} methods. All models are self-improved for one iteration. As seen, HSIR-SFT with varied layer depth can consistently outperform the other baseline methods, indicating that HSIR is relatively robust to the choice of layer. Moreover, when using the middle layer (i.e., 15-th layer), HSIR-SFT achieves the best performance. We conjecture that the middle layer encodes richer and more useful semantic information (Skean et al., 2025; Azaria & Mitchell, 2023; Liu et al., 2019), thus resulting in more accurate *InDiv* scores. Thus, we choose to adopt the middle layer of $\mathcal{M}_{t-1}$ for calculating *InDiv* scores in this work.

More ablation study in *InDiv*. There are two important strategies in our *InDiv*: attention-aware weighting mechanism and *InDiv* normalization. The former is to obtain more fine-grained intrinsic diversity, while the latter is to regularize the *InDiv* score for more flexible data filtering. To evaluate their contributions, we compare our full HSIR-SFT with two variants: 1) “-w/o attention-aware”, which removes the attention-aware weighting mechanism, i.e., $Atten_{i,u}^k = \frac{1}{m}$ in Eq. 5; 2) “-w/o normalization”, which directly uses the Score_i^k in Eq. 5 as the final score and filters the solutions with lower scores within each query. Notably, the filtering ratio is adjusted to ensure that both variants use approximately the same amount of training data as the original HSIR-SFT. Table 11 presents the results, showing that removing either strategy leads to performance degradation. These findings consistently demonstrate the effectiveness of both strategies.

Comparison with Data Selection Baselines. Some readers may wonder how our method relates to prior data selection approaches. While we acknowledge that our method enhances self-improvement performance through better data construction, data selection is not our primary focus. Rather, we address this from two orthogonal perspectives: (1) increasing **data quantity** by eliciting more correct solutions for challenging queries via *VeriExit*, and (2) improving **data quality** by filtering overthinking solutions via *InDiv*. In other words, our work targets fundamental bottlenecks in the self-improvement training of LRMs that are distinct from those addressed by general data selection techniques. To further validate this distinction, we replace the *InDiv* score with several widely used data-centric metrics within our HSIR framework: PPL-based (Cao et al., 2024), Quality-based (Liu et al., 2024), and Entropy-based (Li et al., 2026) filtering. Comparative results on Qwen2.5-1.5B are presented in Table 12. As shown, *InDiv* consistently outperforms all alternatives, demonstrating that it captures a more effective signal for identifying undesired solutions than existing data selection metrics.

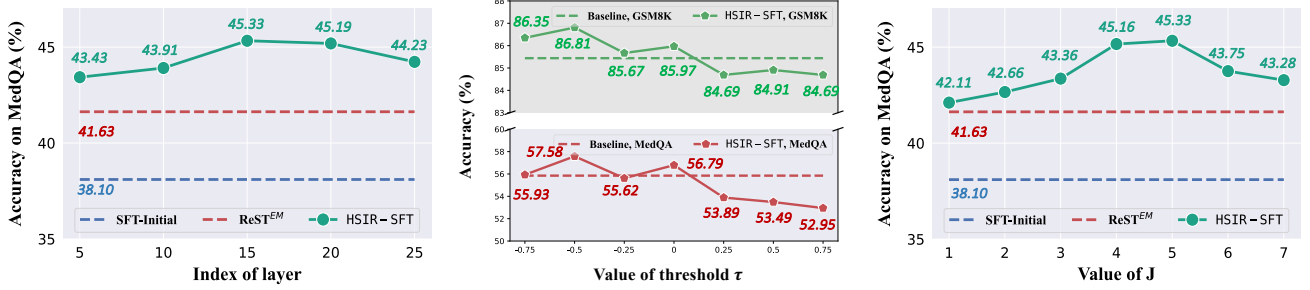


Figure 8. **Left:** Analysis of different layer depths for calculating *InDiv* scores. We use the Qwen2.5-1.5B (with 28 layers) as the test model. **Middle:** Analysis of threshold τ . “Baseline” means that we do not filter the overthinking solutions, *i.e.*, removing the *InDiv*. **Right:** Analysis of threshold τ . “Baseline” means that we do not filter the overthinking solutions, *i.e.*, removing the *InDiv* in HSIR.

D.3. Parameter Analysis

Effect of τ . The threshold τ , used to filter the overthinking solutions, is an important hyperparameter in our HSIR. In this study, we analyze its influence by evaluating the performance with different τ values, spanning from -0.75 to 0.75. Figure 8 (Middle) illustrates the comparative results of Qwen2.5-3B models trained with HSIR-SFT on MedQA and GSM8K. For reference, we also report the results without data filtering as the baseline. As seen, compared to the baseline, HSIR with suitable τ can generally achieve better performance, showing the effectiveness of using *InDiv* scores to filter overthinking solutions. However, too large τ (*i.e.*, 0.75) would lead to performance degradation, as many helpful training samples might be ignored. HSIR performs best with $\tau = -0.5$, thus leaving as our default experimental settings.

Effect of J . The hyperparameter J , which is used to control the sample count of recycled solutions during *VeriExit*, is also important for our HSIR. Here, to investigate its impact, we evaluate our HSIR-SFT with different J values ranging from 1 to 7. The comparative results of Qwen2.5-1.5B on MedQA are illustrated in Figure 8 (Right). For reference, we also include the results of SFT-Initial and ReST^{EM}. All models are self-improved for one iteration. From these results, we find that: 1) When J is too small (*e.g.*, $J = 1$), our *VeriExit* struggles to sample enough correct solutions, limiting the effectiveness of HSIR-SFT. 2) When J is too large (*e.g.*, $J = 7$), many sampled solutions share similar prefix reasoning steps, reducing the diversity of training data and leading to sub-optimal performance. 3) Across all J values, our HSIR-SFT consistently outperforms the baseline methods, proving its robustness. Notably, in the case of $J = 5$, HSIR-SFT achieves the best performance. Thus, we use it as the default setting in this work.

D.4. Impact of Seed Data

Seed data from QWQ-32B. As mentioned in §2.1, we first fine-tune the base model $\mathcal{M}_{base}$ on the seed dataset $\mathcal{S}$ to make it have basic long-CoT reasoning abilities. Intuitively, high-quality seed data can improve the basic reasoning ability of LRMs and boost the effectiveness of self-improvement training. To verify it, we replace the seed data used in our main experiments with that distilled from QWQ-32B⁹. Taking the MedQA as an example, we fine-tune the Qwen2.5 family models on the seed data distilled from QWQ-32B and self-generated pseudo-labeled data for one iteration, using different self-improvement training methods. Table 13 reports the average accuracy and number of output tokens of all tuned models, from which we find that: (1) Compared to the seed data distilled from DeepSeek-R1, the seed data generated by QWQ-32B performs differently in different models. For the smaller models (*i.e.*, Qwen2.5-1.5B), the seed data from QWQ-32B brings more performance gains. Conversely, for the larger 7B model, it leads to worse results. We conjecture that there is a large capacity gap between Qwen2.5-1.5B and DeepSeek-R1, while using a smaller QWQ-32B as a teacher can achieve a smooth knowledge transfer (Mirzadeh et al., 2020; Zhang et al., 2023). However, for the Qwen2.5-7B, a smaller teacher model might struggle to provide sufficient knowledge, thus leading to suboptimal performance. (2) When using the seed data generated by QWQ-32B, our HSIR can still outperform the other baseline methods and achieve better reasoning performance and efficiency. These results demonstrate the universality and robustness of HSIR.

Is the seed data from a frontier model necessary? In the above experiments, we empirically find that initial LRMs trained with high-quality seed data can effectively self-improve via our HSIR. Here, we conduct more in-depth experiments to investigate whether the seed data distilled from a frontier model is necessary. Specifically, we focus on two types of

⁹<https://qwenlm.github.io/blog/qwq-32b-preview/>

Table 13. Performance comparison of the seed data distilled from different LRMs. We evaluate the Qwen2.5 models fine-tuned with different self-improvement SFT methods for one iteration.

Methods	[Train]	Qwen2.5-1.5B		Qwen2.5-3B		Qwen2.5-7B		Overall	
	Avg.	MedQA	Tokens	MedQA	Tokens	MedQA	Tokens	Accuracy	Tokens
<i>Seed data distilled from DeepSeek-R1</i>									
SFT-Initial	1.0K	38.10	1,779	49.02	1,644	62.45	1,428	49.86	1,617
<i>Seed data distilled from QWQ-32B</i>									
SFT-Initial	1.0K	40.22	1,677	50.04	1,569	61.12	1,607	50.46	1,618
SFT-Oracle	8.0K	43.99	1,781	60.25	1,451	69.13	1,419	57.79 $\uparrow 7.33$	1,550 $\downarrow 4.2\%$
STaR	4.9K	39.35	1,576	49.25	1,500	61.82	1,541	50.14 $\downarrow 0.32$	1,539 $\downarrow 4.9\%$
ReST ^{EM}	37.9K	44.38	1,431	54.77	1,307	65.43	1,401	54.86 $\uparrow 4.40$	1,380 $\downarrow 14.7\%$
HSIR-SFT	29.6K	46.58	1,167	55.70	1,192	66.93	1,266	56.40$\uparrow 5.94$	1,208$\downarrow 25.3\%$

Table 14. Analysis of different seed data. We report the MedQA results of tuned Qwen2.5-3B using different self-improvement methods.

Method	Source of seed data			
	DeepSeek	QWQ	Qwen2.5-1.5B	Self-distilled
SFT-Initial	49.02	50.04	42.50	48.70
STaR	49.25	49.25	43.36	47.53
ReST ^{EM}	55.22	54.77	47.18	49.80
HSIR-SFT	55.70	55.70	48.00	50.17

seed data: 1) **low-quality seed data**, which is distilled from a smaller and weaker LRM; 2) **self-distilled seed data**, which is generated by the model itself via an in-context learning approach. Using the Qwen2.5-3B as the testbed, we obtain low-quality seed data from the SFT-Initial Qwen2.5-1.5B. For the self-distilled seed data, we randomly select three examples distilled from DeepSeek-R1 as few-shot demonstrations and use them to guide the base Qwen2.5-3B for generating the seed data. Notably, for all methods, we use the same queries x_i and sample a correct solution for each query, ensuring the same number of training samples across seed data types. Qwen2.5-3B model is first fine-tuned on different seed data, and then self-improved with various SFT approaches for one iteration. Table 14 presents the comparative results of different Qwen2.5-3B models on MedQA. From it, we observe that: 1) the quality of seed data is critical, as initial SFT on low-quality seed data significantly degrades performance; 2) across all seed data types, our HSIR consistently brings performance gains, further validating its effectiveness. In general, while it is feasible for an LLM to self-distill seed data and subsequently self-improve using HSIR, leveraging more high-quality seed data from stronger frontier models allows HSIR to realize its full potential and achieve better performance.

D.5. When Ground-truth Answers are Unavailable

Following many prior studies (Zelikman et al., 2022; Yuan et al., 2023; Wang et al., 2024; Pang et al., 2024), we assume that the ground-truth answers of the unlabeled dataset $\mathcal{D}$ are available in this work. Some readers may wonder how our HSIR performs when ground-truth answers are unavailable in some scenarios. Actually, in this setting, we can follow Huang et al. (2023) and use the majority-voting answer among multiple candidate solutions as a pseudo answer, *i.e.*, $\tilde{y}_i = \arg \max_{\hat{y}_i^j} \sum_{k=1}^K \mathbb{I}(\hat{y}_i^j = \hat{y}_i^k)$. Here, the $\tilde{y}_i$ is denoted as the self-consistency pseudo label. Although the $\tilde{y}_i$ may be incorrect, we can still apply our HSIR to improve the LRMs as described in §3. To verify its effectiveness, we evaluate it on several Qwen2.5 models and report the results in Table 15. For reference, we use the ReST^{EM} as the baseline, and also include the results based on ground-truth answers. As seen, using the self-consistency pseudo labels indeed yields slightly sub-optimal results compared to ground-truth answers. Nevertheless, our HSIR-SFT method still achieves substantial improvements, with an average gain of +6.14%, proving that HSIR remains effective even in unlabeled scenarios.

D.6. Results in High-resource Scenarios

In our work, we assume that only a small amount of seed data is available. Some readers may wonder whether our HSIR method remains effective in high-resource scenarios, where sufficient seed data is provided. To verify it, we use all training

Table 15. Performance comparison of Qwen2.5 models on MedQA and GSM8K benchmarks. Notably, “SC $\rightarrow$ ” refers to using the majority-voting answer among multiple candidate solutions of SFT-Initial models as the pseudo labels of $\mathcal{D}$.

Methods	Qwen2.5-1.5B		Qwen2.5-3B		Qwen2.5-7B		Overall	
	MedQA	GSM8K	MedQA	GSM8K	MedQA	GSM8K	Accuracy	Tokens
SFT-Initial	38.10	63.99	49.02	77.18	62.45	83.93	62.45	1,536
<i>Using ground-truth answers</i>								
ReST ^{EM}	41.63	69.75	55.22	83.95	64.18	88.17	67.15 $\uparrow$ 4.70	1,268 $\downarrow$ 17.5%
HSIR-SFT	45.33	71.72	55.70	86.13	67.32	88.78	69.16 $\uparrow$ 6.71	1,075 $\downarrow$ 30.0%
<i>Using self-consistency pseudo labels</i>								
SC $\rightarrow$ ReST ^{EM}	40.46	70.36	51.69	83.70	64.57	87.04	66.30 $\uparrow$ 3.85	1,322 $\downarrow$ 13.9%
SC $\rightarrow$ HSIR-SFT	44.78	71.72	54.99	84.69	65.91	89.46	68.59 $\downarrow$ 6.14	1,108 $\downarrow$ 27.8%

Table 16. Results in high-resource scenarios. The SFT-Oracle is used as the initial $\mathcal{M}_0$, which is trained with self-improvement methods.

Method	Qwen2.5-1.5B		Qwen2.5-3B	
	MedQA	Tokens	MedQA	Tokens
SFT-Oracle	46.58	1,678	58.68	1,448
<i>Using SFT-Oracle as the initial model $\mathcal{M}_0$</i>				
ReST ^{EM}	47.21 $\uparrow$ 0.63	1,423 $\downarrow$ 15.2%	61.81 $\uparrow$ 3.13	1,243 $\downarrow$ 14.2%
HSIR-SFT	50.90 $\uparrow$ 4.32	1,156 $\downarrow$ 31.1%	63.71 $\uparrow$ 5.03	1,116 $\downarrow$ 22.9%
IRPO	46.35 $\downarrow$ 0.23	1,683 $\uparrow$ 0.3%	59.46 $\uparrow$ 0.78	1,585 $\uparrow$ 9.5%
HSIR-DPO	53.49 $\uparrow$ 6.91	1,047 $\downarrow$ 37.6%	66.61 $\uparrow$ 7.93	1,116 $\downarrow$ 22.9%

samples distilled from DeepSeek-R1 as the seed data to initially fine-tune the base model, *i.e.*, using the SFT-Oracle in Table 1 as the $\mathcal{M}_0$. Table 16 presents the results of Qwen2.5-1.5B and Qwen2.5-3B models on MedQA. For reference, we also report the results of ReST^{EM} and IRPO as baselines. All models are self-improved for one iteration. From these results, we find that both HSIR-SFT and HSIR-DPO methods can effectively improve the performance and inference efficiency of SFT-Oracle models. Specifically, with the help of HSIR-DPO, Qwen2.5-3B model achieves +7.93% performance gain on MedQA. These findings demonstrate that our HSIR has great potential to enhance the self-improvement capabilities of fully-trained and powerful LRMs.

D.7. More Self-improvement Iterations

Due to limited computational resources, we set the maximum self-improvement iterations T to 3 in the main experiments. Here, to further investigate whether additional iterations can improve performance, we extended the maximum training iterations T from 3 to 5, and compare the MedQA accuracy of Qwen2.5-1.5B models trained with different self-improvement SFT training methods across the iterations. Figure 9 (Left) illustrates the comparative results, from which we observe that: 1) With the increase of self-improvement training iterations, both STaR and ReST^{EM} exhibit a trend where performance initially improves but then declines, which is similar to the findings of Ding et al. (2025). This may be due to overfitting on easy-to-learn samples. Conversely, by mitigating the data imbalance problem, our HSIR can collect more challenging samples and achieve continuous performance improvements. 2) As self-improvement training progresses, the performance gains of HSIR tend to be smaller, indicating the existence of an upper-bound for self-improvement training.

D.8. Comparison and Compatibility with Inference-time Methods

The goal of our work is to propose a self-improvement training framework that unlocks the internal long-CoT reasoning capabilities of LRMs, rather than to optimize inference. Therefore, in the main experiments, we do not compare HSIR with inference-time methods, such as Self-Consistency (SC) (Wang et al., 2023). Nevertheless, given that SC is widely used to enhance LRM reasoning performance, we include it in this experiment. Specifically, during inference, we sample five solutions from the model and select the majority-vote answer as the prediction. Figure 9 (Right) shows the comparative results of Qwen2.5 models on MedQA, where “-w/ SC” means using the SC method. As seen, increasing the test-time compute through SC improves the reasoning performance of SFT-Initial models effectively. However, it still underperforms

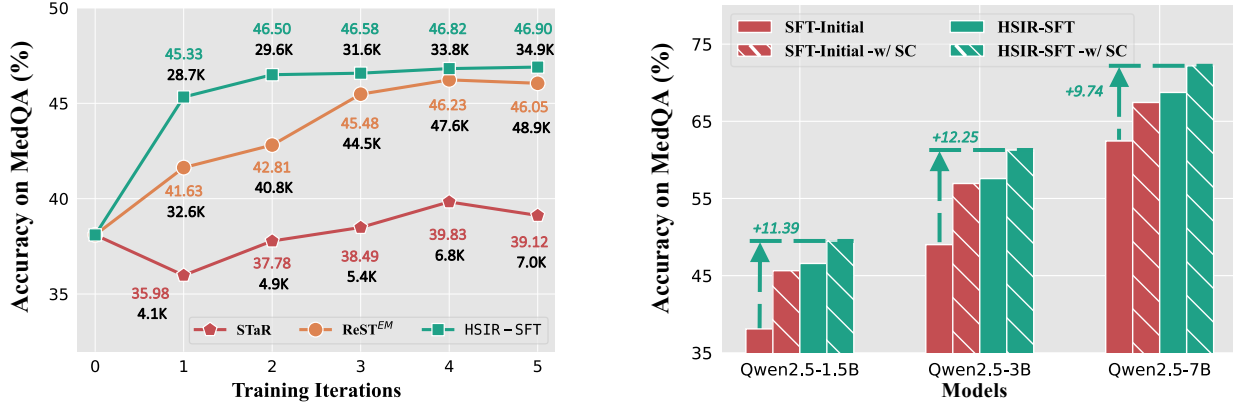


Figure 9. **Left:** Results of Qwen2.5-1.5B models training for more self-improvement iterations. Here, we report both test accuracy and the number of training samples on MedQA. **Right:** Performance comparison between with and without the self-consistency method. Notably, we report the results of HSIR-SFT after three self-improvement iterations.

our HSIR-SFT method, even if HSIR-SFT only samples a single solution during inference. More encouragingly, combining the HSIR-SFT and SC methods consistently yields further performance improvements. For instance, for the Qwen2.5-3B model, with the help of SC, HSIR-SFT achieves a 12.25% performance gain over the SFT-Initial model. These results prove the compatibility of our HSIR with the inference-time SC method.

D.9. Efficiency of HSIR

Some readers may be concerned about the efficiency of our HSIR method, as it requires additional forward passes of LRMs. Actually, during the *VeriExit* sampling phase, we only sample the final answer without regenerating intermediate reasoning trajectories, making it much faster than simply allocating more trials to failed queries. In our preliminary experiments, we found that the *VeriExit* sampling can be completed in an average of one hour on 8 NVIDIA A800 (80GB) GPUs, which is about 1/4 of the time required for generating full reasoning trajectories. On the other hand, to obtain the *InDiv* scores, the query and its solution are fed into the model in a teacher-forcing manner, requiring only a single forward pass and introducing minimal latency. In practice, during HSIR-SFT and HSIR-DPO training, obtaining hidden states for each token introduces some computational overhead, but this can be completed in about half an hour on 8 NVIDIA A800 (80GB) GPUs. Moreover, during the H-GRPO training, we can reuse the hidden representation obtained by the reference model without extra forward passes. The actual computation of the *InDiv* score itself is lightweight, involving only simple vector operations that take a few seconds. More importantly, owing to the data filtering process of HSIR, the training budget can be significantly reduced, *e.g.*, from 53.6K to 38.8K during fine-tuning Qwen2.5-1.5B at the last iteration. In general, the inference latency of HSIR is tolerable against its performance gains.

D.10. Case Study

To better understand our methods and prove their effectiveness, we provide some cases and perform the qualitative analyses in this part. Specifically, Table 17 shows the comparison of self-generated solutions between before and after using our *VeriExit* sampling strategy. We show the self-generated results of Qwen2.5-1.5B on the MedQA task. As seen in the previous failed solutions, the model indeed arrived at the correct answer during intermediate reasoning steps, yet ultimately failed to produce the accurate outcome due to reasoning deviation. Conversely, by using our verify-then-exit strategy, we can convert it into a collect solution. Table 18 shows the comparison of *InDiv* scores between concise and overthinking self-generated solutions. Obviously, *InDiv* scores can reflect the redundancy and repetitiveness of reasoning steps generated by Qwen2.5-1.5B on the MedQA task. We observe that the overthinking solution indeed obtains a much lower *InDiv* score. These case studies support the intuition and soundness of our proposed methods.

Table 19 and Table 20 show the comparison of reasoning results of Qwen2.5-7B models before and after training using HSIR on MedQA and GSM8K test sets, respectively. It can be found that, with the help of HSIR, the model can reason more accurately and concisely, thus resulting in better performance. These case studies prove the effectiveness of HSIR.

Table 17. An example of converting the failed solution into the correct one via the *VeriExit* strategy. In this case from MedQA, the ground-truth answer is “D”. As seen, the failed solution arrived at the correct answer during intermediate reasoning steps (highlighted in green), but diverted into an incorrect one due to overly backtracking (highlighted in red).

Previous Failed Solution:

<think>

Okay, let’s try to work through this question.

...

So, if the options include allopurinol (option C), then that’s the answer. But the options given here are D. Xanthine oxidase inhibitor. Wait, but xanthine oxidase inhibitors are allopurinol and other drugs like febuxostat. So, allopurinol is a xanthine oxidase inhibitor. So, if the answer is C, then allopurinol. But the question is asking for the long-term treatment. So, if the options don’t include allopurinol, then perhaps the answer is D. Wait, I think the answer is C. Allopurinol. So, the correct answer would be C.

...

##Answer:

C

</answer>

New Solution Generated by *VeriExit*:

<think>

Okay, let’s try to work through this question.

...

So, if the options include allopurinol (option C), then that’s the answer. But the options given here are D. Xanthine oxidase inhibitor. Wait, but xanthine oxidase inhibitors are allopurinol and other drugs like febuxostat. So, allopurinol is a xanthine oxidase inhibitor. So, if the answer is C, then allopurinol. But the question is asking for the long-term treatment. So, if the options don’t include allopurinol, then perhaps the answer is D.

</think> (verify then exit)

<answer>

...

##Answer:

D

</answer>

Table 18. Comparison of *InDiv* scores between concise and overthinking solutions in a single query. Here, we use the case of MedQA. Compared to the concise solution, the overthinking solution contains more repetitive reasoning steps (highlighted in orange).

Concise Solution:

<think>

Okay, let’s try to figure this out. So the question is about malnutrition in children and which amino acid is needed for them. The symptoms described are Kwashiorkor, which is protein deficiency. The options are A. Alanine, B. Tyrosine, C. Arginine, D. Methionine.

First, I remember that kwashiorkor is typically due to inadequate protein intake. The main issue is the lack of essential amino acids, especially the ones that the body can’t make on its own. The nine essential amino acids are tryptophan, threonine, histidine, lysine, isoleucine, leucine, methionine, phenylalanine, and valine.

Looking at the options, Methionine (D) is one of those essential amino acids. Alanine (A) is a non-essential one, so maybe not needed here. Tyrosine (B) is an amino that’s non-essential and involved in protein synthesis but not for this presentation. Arginine (C) is also a non-essential one and more associated with metabolic pathways like making creatine or proteins, not really for this symptom. So the answer should be D. Methionine.

</think>

...

</answer>

InDiv Score: 1 . 32

Overthinking Solution:

<think>

Okay, let's tackle this question.

...

So the answer would be D. Methionine. Wait, but I thought Tyrosine is an essential amino acid ...

So perhaps the answer is D. Methionine, because in some cases, the body uses up all the available EAA's ...

So the answer is D. Methionine. Because in kwashiorkor, without proper nutrition, the body cannot utilize other amino acids. So supplementing with Methionine would help ...

Therefore, the correct answer is D. Methionine. Because even if the diet has methionine, the body can't utilize it. ...

Alternatively, maybe the answer is Tyrosine. But I think not. So the answer is D.

</think>

...

</answer>

InDiv Score: -0 . 83

Table 19. Comparison of MedQA reasoning results of Qwen2.5-7B models before and after training using HSIR. The ground-truth answer of this case is "D".

Instruction:

For the following medical question, select one correct answer from A to D.

Question: A 25-year-old African American man presents to his primary care provider for routine blood work. He is a well-known case of sickle cell disease treated with hydroxyurea and tramadol as needed for pain. He does not smoke or drink alcohol. The patient asks if his disease can adversely affect his kidneys because his friend was recently diagnosed with end-stage renal failure. Which of the following is a renal complication of sickle cell disease?

Options:

- A. Membranous glomerulonephritis
- B. Proximal renal tubular acidosis
- C. Impaired urine diluting ability
- D. Impaired urine concentration ability

Output of SFT-Initial Model:

<think>

Okay, let's tackle this question. So, the patient is a 25-year-old African American man with sickle cell disease. He's on hydroxyurea and tramadol for pain. His friend has end-stage renal failure, and he's asking if his disease can affect his kidneys. The question is about a renal complication of sickle cell disease. The options are A to D.

First, I need to recall the common renal complications associated with sickle cell disease. Sickle cell disease can lead to various kidney issues. Let me think. The kidneys are affected because of the hemolytic anemia and the high levels of hemoglobin S, which can cause microangiopathic hemolysis.

Membranous glomerulonephritis (option A) is a type of nephrotic syndrome. I remember that in sickle cell disease, membranous nephropathy is a known complication. It's an immune complex-mediated process, leading to proteinuria and nephrotic syndrome. So that's a possibility.

Proximal renal tubular acidosis (option B) is a genetic disorder where the kidneys can't excrete acids properly. But I don't think that's linked to sickle cell. PRSA is more about a defect in the renal tubules' ability to handle acid, which isn't directly related to sickle cell's hemolysis.

Impaired urine diluting ability (option C) would be nephrogenic diabetes insipidus. That's when the kidneys can't concentrate urine due to a defect in response to ADH. Again, not directly linked to sickle cell. Maybe from other causes like certain drugs or genetic disorders.

Impaired urine concentration ability (option D) is nephrogenic diabetes insipidus as well. So this is the same as option C. **So D is not the right answer here.**

So the options are between A and C. Wait, the question is about a renal complication. Sickle cell can cause various issues. Let me think again. The patient's friend had end-stage renal failure, which could be due to chronic kidney disease from sickle cell. The options given are specific conditions.

Membranous glomerulonephritis (A) is a type of glomerular disease leading to nephrotic syndrome. In sickle cell, membranous nephropathy is a recognized complication, often presenting with nephrotic syndrome. So A is a possible answer.

Impaired urine diluting ability (C) would be nephrogenic DI, which is unrelated. **So the correct answer here is A. Membranous glomerulonephritis.** </think>

<answer>

...

##Answer:

A

</answer>

Output of HSIR-DPO Model:

<think>

Okay, let's tackle this question. So, the patient is a 25-year-old African American man with sickle cell disease. He's on hydroxyurea and tramadol for pain. His friend had end-stage renal failure, and the patient wants to know if his disease can affect his kidneys. The options are about different renal complications.

First, I need to recall the common renal issues associated with sickle cell disease. Sickle cell disease can cause various complications, including kidney problems. I remember that sickle cell can lead to different glomerular diseases. For example, sickle cell nephropathy, which is a type of glomerulonephritis. But looking at the options, option A is membranous glomerulonephritis. I think that's more commonly associated with conditions like lupus or hepatitis B, but maybe I'm mixing up. Alternatively, maybe it's a rare complication.

Wait, another possibility is that sickle cell can cause tubular issues. The options include proximal renal tubular acidosis (option B). I know that sickle cell can lead to Fanconi syndrome, which is a form of proximal tubular dysfunction. Fanconi syndrome includes glucosuria, aminoaciduria, etc. But option B is specifically proximal RTA, which is a different condition. Proximal RTA is when the proximal tubule can't reabsorb properly, leading to loss of nutrients and acidosis. But I'm not sure if that's directly linked to sickle cell.

Alternatively, the question might be pointing towards glomerular issues. The options include impaired urine diluting ability (option C) and impaired concentration ability (option D). I remember that in sickle cell disease, there's a condition called hypernephrosis, which is an increase in kidney size due to fluid overload. This can lead to dilutional hyponatremia because the kidneys can't concentrate urine properly. **So, impaired concentration ability (option D) would be the issue here.** But wait, the options are C and D as separate. Wait, impaired urine diluting ability—so the kidneys can't excrete concentrated urine. Impaired concentration ability would be the opposite, right? So in sickle cell, the kidneys might have trouble concentrating urine, leading to dilutional hyponatremia. So option D would be the correct answer here.

But wait, another thought: sickle cell can cause glomerulonephritis leading to proteinuria, hematuria, etc. But the options don't mention that. The options given are specific. **So, if the question is about a renal complication, maybe the answer is D.**

</think>
 <answer>
 ...
 ##Answer:
 D
 </answer>

Table 20. Comparison of GSM8K reasoning results of Qwen2.5-7B models before and after training using HSIR. The ground-truth answer of this case is “20”.

Instruction:

Toby is reading a book that is 45 pages long. It averages 200 words a page. Toby can read at a rate of 300 words per minute. He has to be at the airport in 60 minutes and plans to leave as soon as he finishes the book. It takes 10 minutes to get to the airport. How many minutes early will Toby be?

Output of SFT-Initial Model:

<think>
 Okay, let’s see. So the problem is about Toby reading a book and figuring out how early he’ll be at the airport. Hmm. Let me start by understanding each part step by step.
 First, the book is 45 pages long. Each page averages 200 words. So, to find the total number of words in the book, I need to multiply the number of pages by the average words per page. That would be 45 pages times 200 words per page. Let me calculate that. 45 times 200... 45 times 200 is 9,000 words. So the book has 9,000 words in total.
 Next, Toby’s reading rate is 300 words per minute. So, to find out how long it takes him to read the entire book, I need to divide the total number of words by his reading rate. That’s 9,000 words divided by 300 words per minute. Let me do that division. 9,000 divided by 300. Hmm, 300 times 30 is 9,000. So that’s 30 minutes. So Toby will take 30 minutes to read the book.
 But wait, he has to be at the airport in 60 minutes, and it takes him 10 minutes to get there. **So the total time he needs to leave his house is 60 minutes (time to be at the airport) plus 10 minutes (travel time) which is 70 minutes.** But he only needs 30 minutes to read the book. So if he starts reading now, he’ll finish in 30 minutes. Then he has $70 - 30 = 40$ minutes left before he needs to leave. Therefore, he’ll be 40 minutes early.
 Wait, but let me check again to make sure I didn’t miss anything. Let me recap: Total words in the book: 45 pages * 200 words/page = 9,000 words. Reading time: 9,000 words / 300 words per minute = 30 minutes. Time needed to get to the airport: 10 minutes. Total time before he needs to leave: 60 minutes.
 So time he has after finishing reading: $60 + 10 - 30 = 40$ minutes. Therefore, he’s 40 minutes early. That seems right. I don’t think I made any mistakes here. The key steps were calculating the total words, then the reading time, and then subtracting that from the total time he has before he needs to leave. Yeah, that makes sense.
 </think>
 ...
 Final Answer: \boxed{40}

Output of HSIR-DPO Model:

<think>
 Okay, let’s see. Toby has a 45-page book to read. Each page averages 200 words. He reads at 300 words per minute. He has 60 minutes before he needs to be at the airport, and it takes him 10 minutes to get there. The question is how many minutes early he’ll be if he reads the whole book and leaves right after. Hmm, let me break this down step by step.

First, I need to figure out how many words are in the entire book. Since there are 45 pages and each page has 200 words, I can multiply those two numbers. So 45 pages times 200 words per page. Let me calculate that. 45 times 200... 45 times 200 is 9,000 words. So the book has 9,000 words total.

Next, Toby reads at a rate of 300 words per minute. To find out how long it takes him to read the entire book, I need to divide the total number of words by his reading rate. That's 9,000 words divided by 300 words per minute. Let me do that division. $9,000 \div 300$. Hmm, 300 times 30 is 9,000. So that's 30 minutes. So reading the book takes him 30 minutes.

But he has to get to the airport, which takes 10 minutes. So the total time from when he starts reading until he leaves for the airport is the time reading plus the time to get there. That's 30 minutes + 10 minutes = 40 minutes. Wait, but he has 60 minutes before he needs to be at the airport. So if he finishes reading and leaves 40 minutes later, he'll have $60 - 40 = 20$ minutes left. That means he'll be 20 minutes early.

Let me just verify each step again. Pages times words per page: $45 \times 200 = 9000$. Reading time: $9000 / 300 = 30$. Time to airport: 10. Total time: $30 + 10 = 40$. Time left: $60 - 40 = 20$. Yep, that checks out. So the answer is 20 minutes early.

</think>

...

****Final Answer****: \boxed{20}
